4장

분할 정복 기법

DIVIDE AND CONQUER

큰 문제를 잘게 쪼개어 정복하고, 다시 하나로 합친다

이번 장에서 배울 내용

4.1 분할 정복의 기본

4.2 병합 정렬 (Merge Sort)

4.3 퀵 정렬 (Quick Sort)

4.4 최대 구간 합

4.5 최근접 쌍

4.6 쿼드 트리

4.7 마스터 정리

4.8 카라츠바 알고리즘

4.9 스트라센 알고리즘

4.10 코딩 테스트 예제

4.1

분할 정복이란?

"나누어라, 그리고 정복하라"
큰 문제를 작은 문제로 쪼개어 해결하는 알고리즘 설계 전략

나누어라, 그리고 정복하라

큰 문제를 같은 유형의 작은 문제 여러 개로 나누어 각각 해결합니다. 작은 문제의 답을 결합해 원래 문제의 답을 만듭니다.

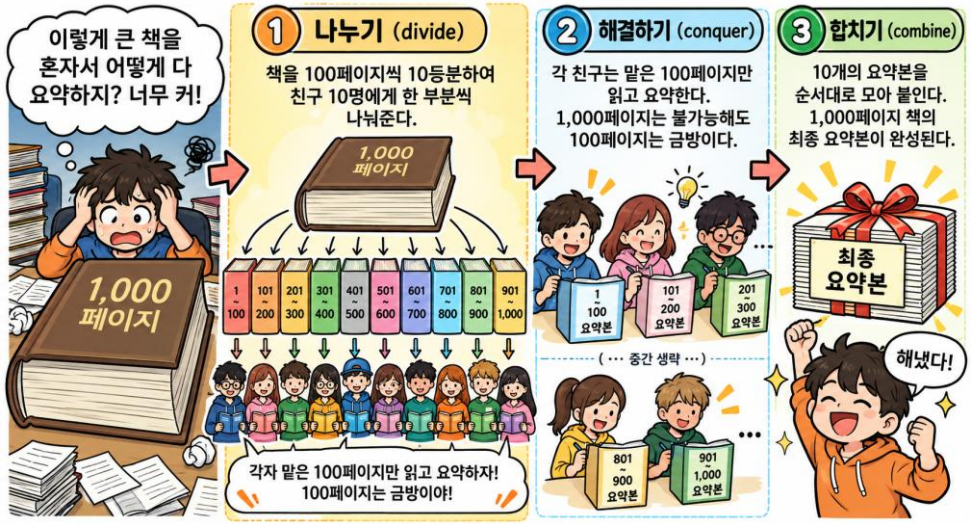
분할, 정복, 결합의 구조를 재귀적으로 적용합니다.

Divide → Conquer → Combine



직관적 이해 — 거대한 책 요약하기

1,000페이지짜리 책을 혼자 요약하려면 막막하다. 하지만 친구 10명이 있다면 이야기가 달라진다.



나누기 (divide)

책을 100페이지씩 10등분하여 친구 10명에게 한 부분씩 나눠준다.

해결하기 (conquer)

각 친구는 맡은 100페이지만 읽고 요약한다. 1,000페이지는 불가능해도 100페이지는 금방이다.

합치기 (combine)

10개의 요약본을 순서대로 모아 붙인다. 1,000페이지 책의 최종 요약본이 완성된다.

큰 문제를 그대로 상대하면 어렵지만, 작게 쪼개면 각각은 쉽다 — 이것이 분할 정복의 핵심 통찰이다.

분할 정복의 3단계

분할 (Divide)

문제를 같은 유형의 더 작은 하위 문제 여러 개로 나눈다.

정복 (Conquer)

하위 문제를 재귀적으로 해결한다. 충분히 작아지면 직접 푼다.

결합 (Combine)

하위 문제의 해답을 조합해 원래 문제의 해답을 만든다.

분할 정복은 한 번만 나누는 알고리즘이 아니다

분할된 문제가 아직 커서 즉시 해결할 수 없다면 다시 분할한다. 아주 쉽게 풀릴 때까지 계속 분할한다.

divide_and_conquer.pseudo

```
DIVIDE_AND_CONQUER(P):
    if P의 크기가 충분히 작으면 then return SolveDirectly(P)
    P1, P2, ..., Pk <- Divide(P)                  // 분할
    Si <- DIVIDE_AND_CONQUER(Pi)   for i = 1..k   // 정복
    return Combine(S1, S2, ..., Sk)               // 결합
```

병합 정렬 (Merge Sort)

반으로 나누고, 정렬된 두 배열을 합친다
원소가 하나이면 이미 정렬된 상태다.

정렬 전

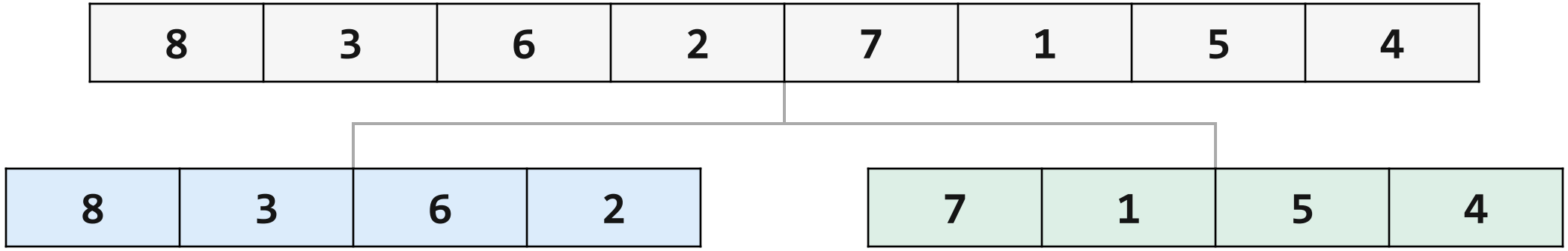
8	3	6	2	7	1	5	4

정렬 후

1	2	3	4	5	6	7	8

나누기 ① · 8개를 4개씩

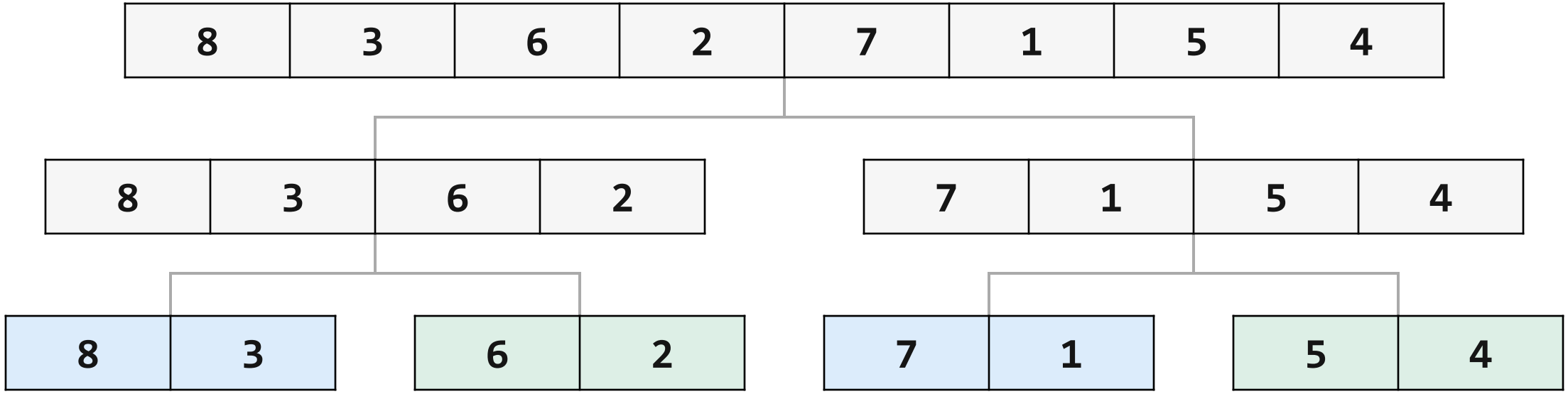
가운데 위치에서 나눕니다. 숫자의 크기는 아직 비교하지 않습니다.
왼쪽 [8,3,6,2], 오른쪽 [7,1,5,4]의 원래 순서를 유지합니다.



다음: 네 개짜리 구간도 더 작은 문제로 나눕니다.

나누기 ② · 4개를 2개씩

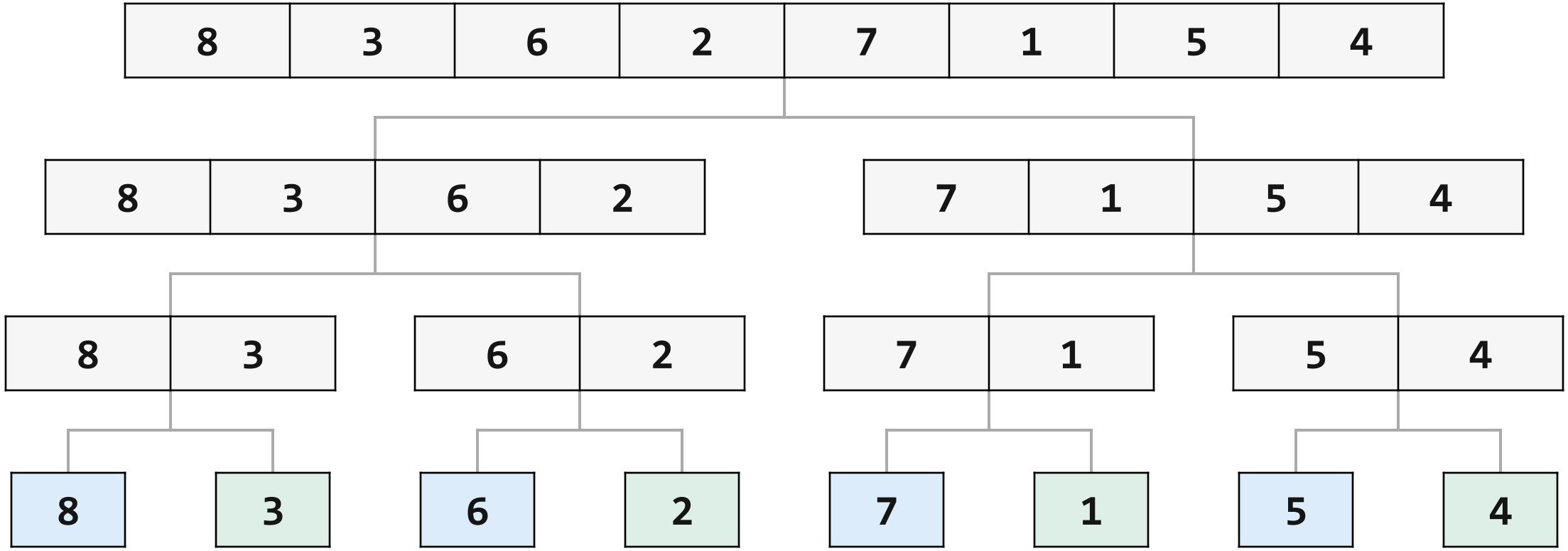
네 개짜리 구간도 각각 가운데에서 나눕니다.
두 개짜리 네 묶음이 생겼지만, 아직 정렬된 결과는 아닙니다.



다음: 두 개짜리 구간을 원소 하나씩 나눕니다.

나누기 ③ · 2개를 1개씩

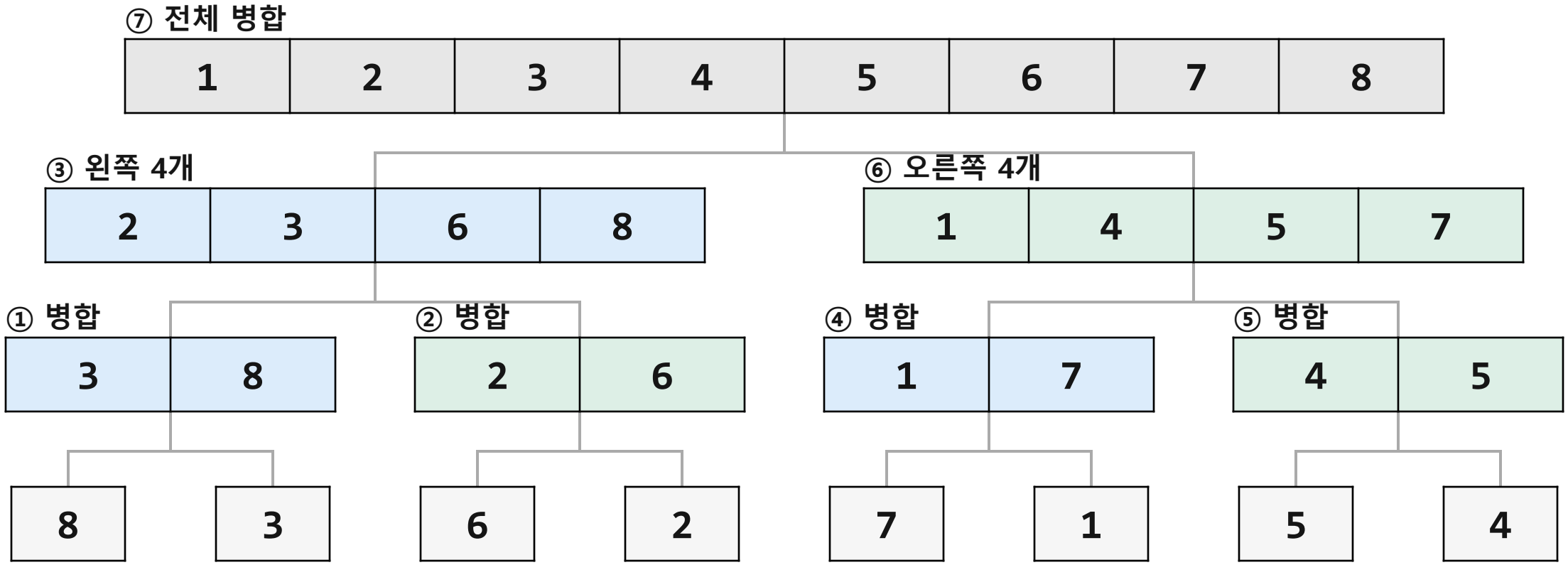
원소가 하나이면 더 정렬할 것이 없습니다. 여기서 재귀 호출을 멈춥니다.
이제 한 원소짜리 결과 두 개를 받아 병합을 시작합니다.



한 원소 = 이미 정렬됨 → 반환 → 두 결과를 병합

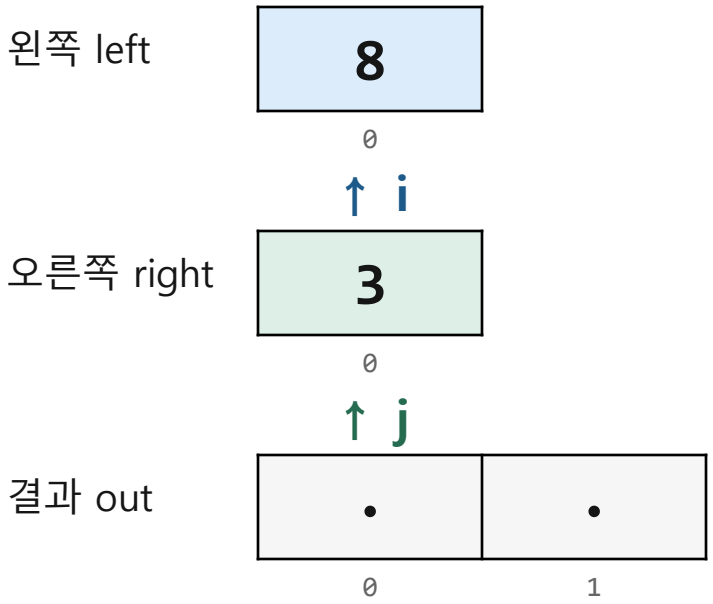
실제 병합 순서 · 왼쪽부터 끝낸다

① → ② → ③: 왼쪽 4개 완료 ④ → ⑤ → ⑥: 오른쪽 4개 완료
⑦: 두 결과를 합쳐 전체 8개 정렬

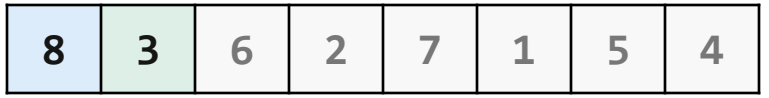


① [8]과 [3] 병합 · 시작

i = 0, j = 0. 왼쪽 8과 오른쪽 3을 비교합니다.
정렬된 두 결과를 받아 작은 값부터 새 결과에 넣습니다.



처음 배열에서 이번에 합치는 부분



왼쪽 4개의 앞쪽 두 개

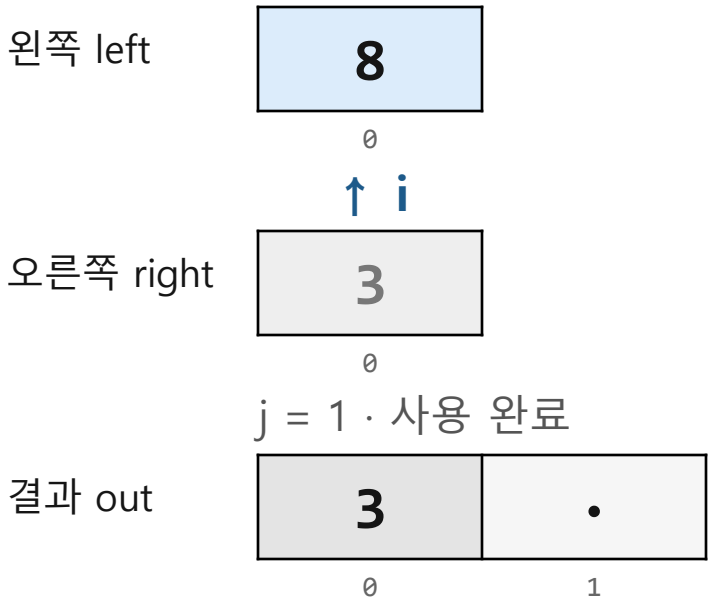
한 원소짜리 두 결과를 받습니다.

선택한 쪽의 인덱스만
한 칸 움직입니다.

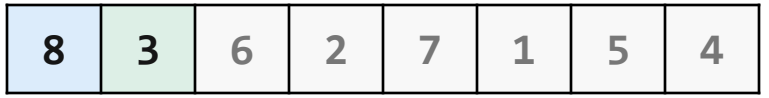
파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

① 병합 · 1번째 값 3 추가

왼쪽 8 > 오른쪽 3 → 오른쪽의 3을 붙입니다.
j는 0 → 1. 사용하지 않은 쪽의 인덱스는 그대로입니다.



처음 배열에서 이번에 합치는 부분



왼쪽 4개의 앞쪽 두 개

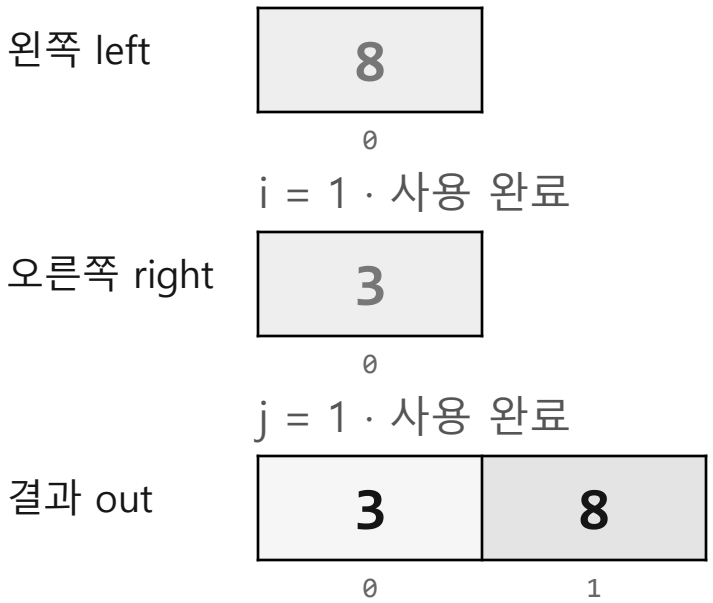
한 원소짜리 두 결과를 받습니다.

선택한 쪽의 인덱스만
한 칸 움직입니다.

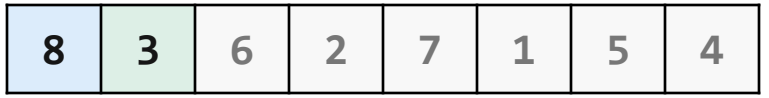
파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

① 병합 · 2번째 값 8 추가

오른쪽을 다 사용했습니다. 남은 왼쪽의 8을 붙입니다.
i는 0 → 1. 이번 병합의 정렬 결과가 완성되었습니다.



처음 배열에서 이번에 합치는 부분



왼쪽 4개의 앞쪽 두 개

이번 병합의 정렬 결과

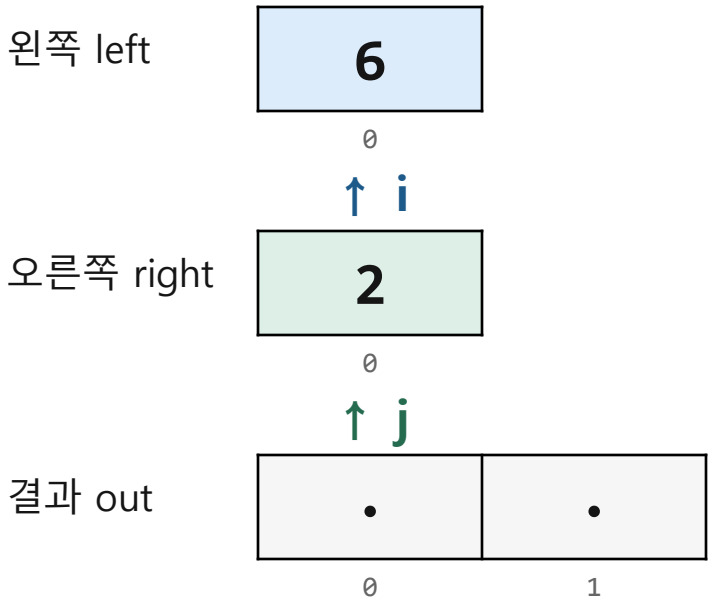
[3, 8]

③에서 왼쪽 입력으로 사용

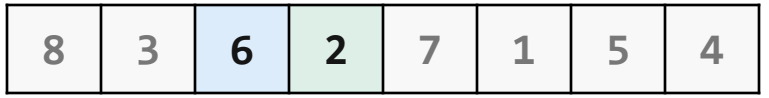
파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

② [6]과 [2] 병합 · 시작

i = 0, j = 0. 왼쪽 6과 오른쪽 2을 비교합니다.
정렬된 두 결과를 받아 작은 값부터 새 결과에 넣습니다.



처음 배열에서 이번에 합치는 부분



왼쪽 4개의 뒤쪽 두 개

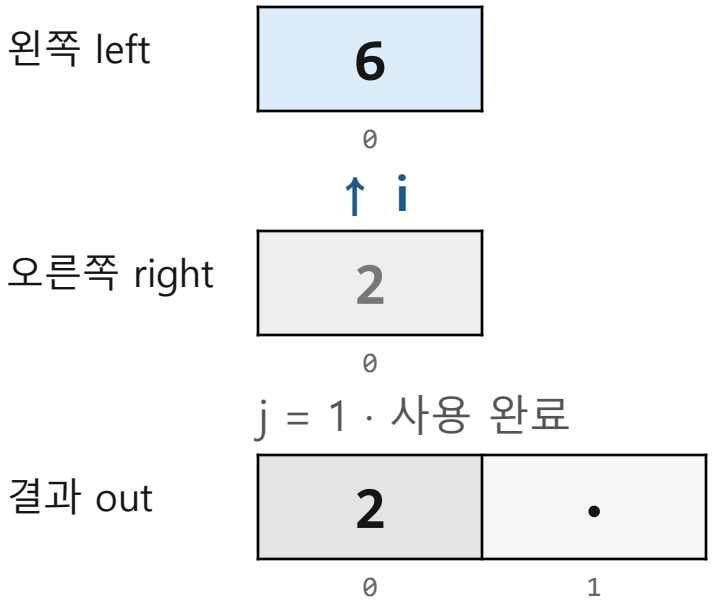
①의 [3,8]은 준비된 채 기다립니다.

선택한 쪽의 인덱스만
한 칸 움직입니다.

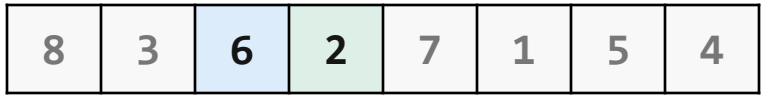
파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

② 병합 · 1번째 값 2 추가

왼쪽 6 > 오른쪽 2 → 오른쪽의 2을 붙입니다.
j는 0 → 1. 사용하지 않은 쪽의 인덱스는 그대로입니다.



처음 배열에서 이번에 합치는 부분



왼쪽 4개의 뒤쪽 두 개

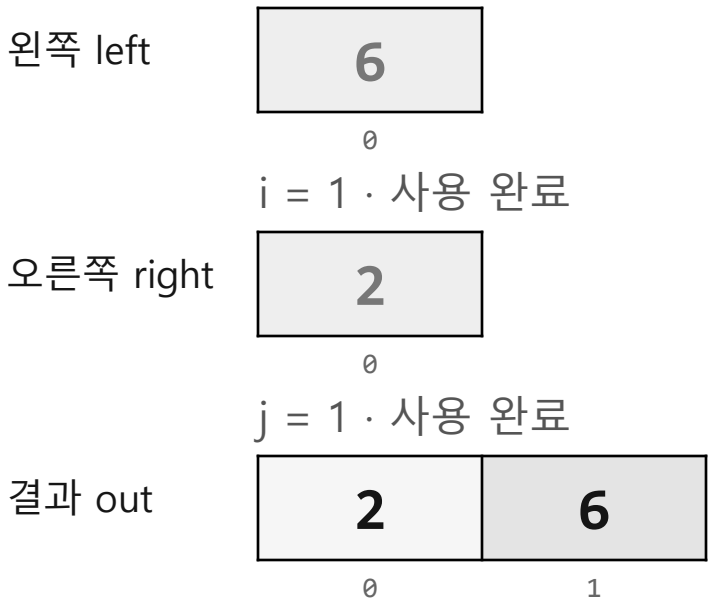
①의 [3,8]은 준비된 채 기다립니다.

선택한 쪽의 인덱스만
한 칸 움직입니다.

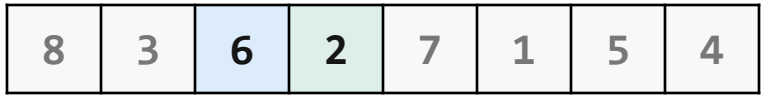
파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

② 병합 · 2번째 값 6 추가

오른쪽을 다 사용했습니다. 남은 왼쪽의 6을 붙입니다.
i는 0 → 1. 이번 병합의 정렬 결과가 완성되었습니다.



처음 배열에서 이번에 합치는 부분



왼쪽 4개의 뒤쪽 두 개

이번 병합의 정렬 결과

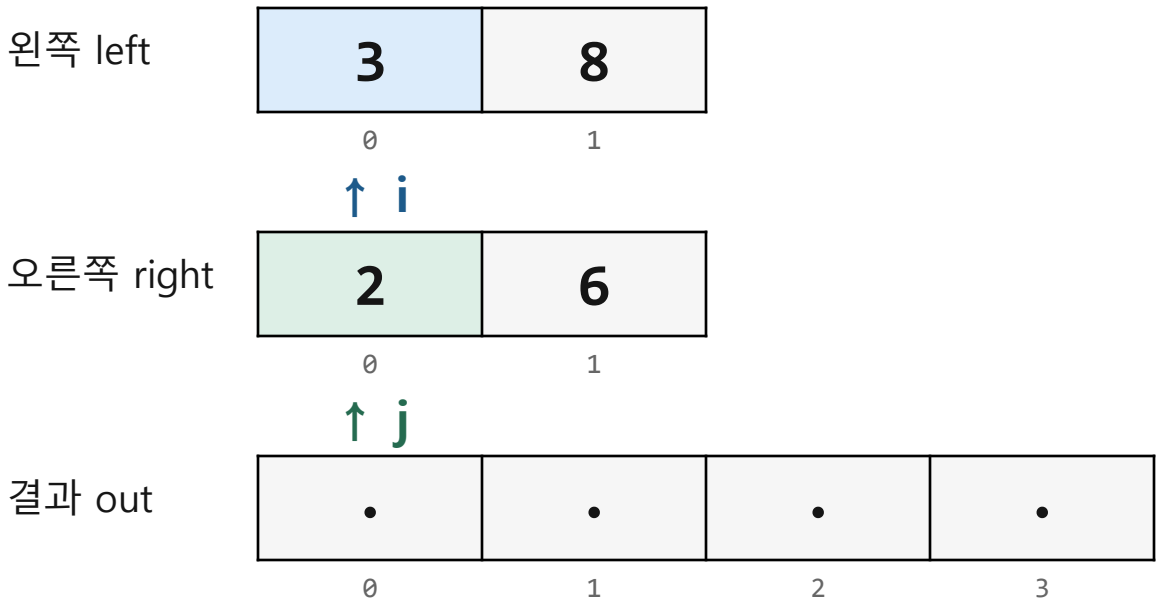
[2, 6]

③에서 오른쪽 입력으로 사용

파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

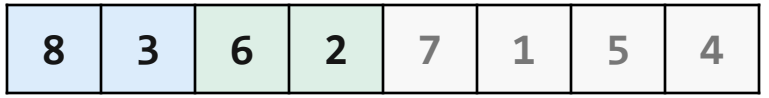
③ [3, 8]과 [2, 6] 병합 · 시작

i = 0, j = 0. 왼쪽 3과 오른쪽 2을 비교합니다.
정렬된 두 결과를 받아 작은 값부터 새 결과에 넣습니다.



파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

처음 배열에서 이번에 합치는 부분



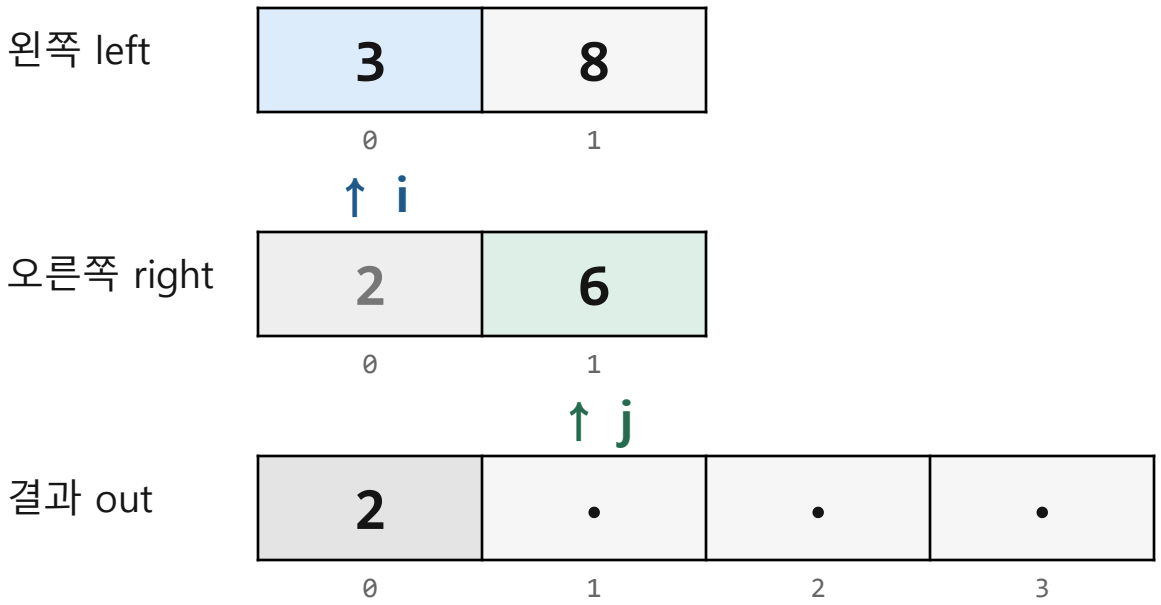
왼쪽 4개 전체

왼쪽: ①의 결과 오른쪽: ②의 결과

선택한 쪽의 인덱스만
한 칸 움직입니다.

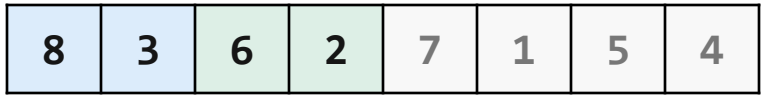
③ 병합 · 1번째 값 2 추가

왼쪽 3 > 오른쪽 2 → 오른쪽의 2을 붙입니다.
j는 0 → 1. 사용하지 않은 쪽의 인덱스는 그대로입니다.



파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

처음 배열에서 이번에 합치는 부분



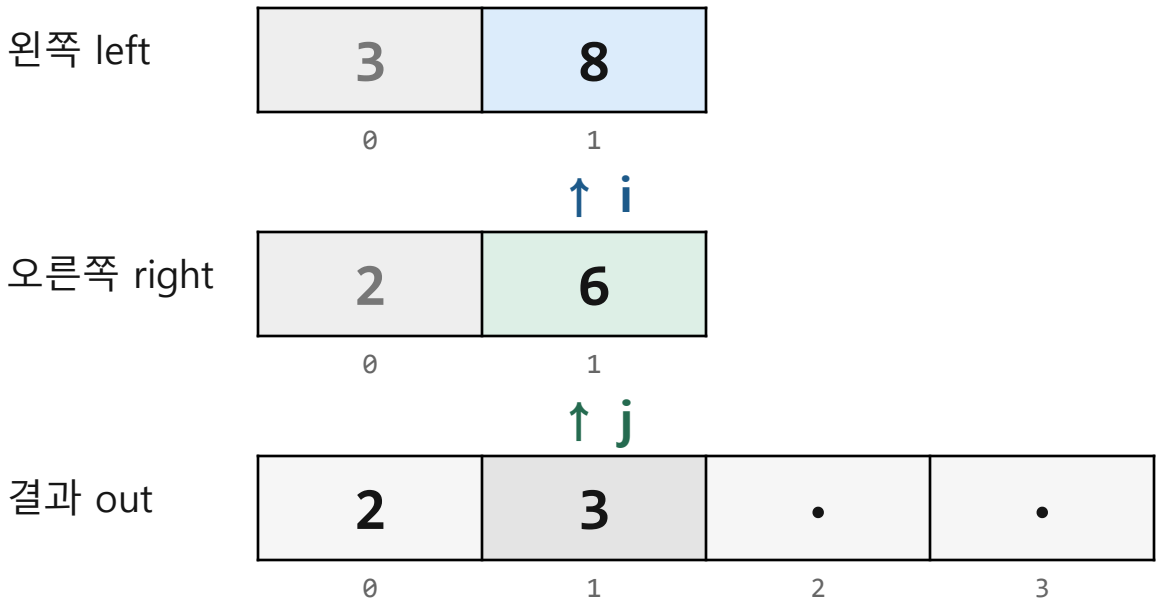
왼쪽 4개 전체

왼쪽: ①의 결과 오른쪽: ②의 결과

선택한 쪽의 인덱스만
한 칸 움직입니다.

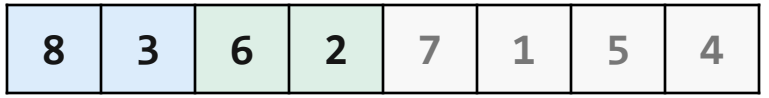
③ 병합 · 2번째 값 3 추가

왼쪽 3 ≤ 오른쪽 6 → 왼쪽의 3을 붙입니다.
i는 0 → 1. 사용하지 않은 쪽의 인덱스는 그대로입니다.



파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

처음 배열에서 이번에 합치는 부분



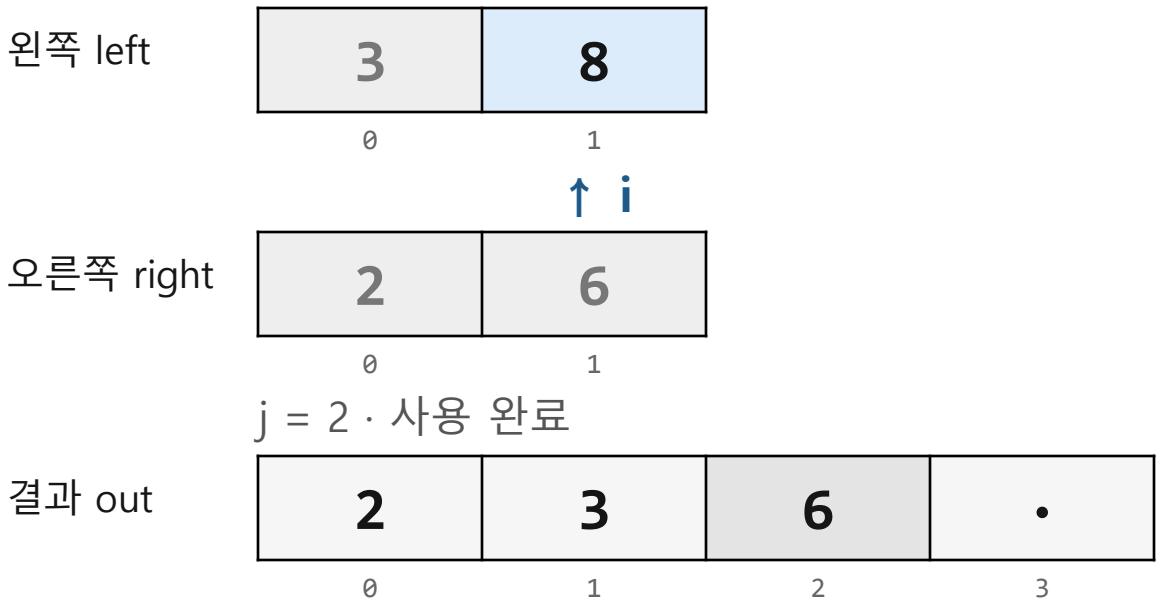
왼쪽 4개 전체

왼쪽: ①의 결과 오른쪽: ②의 결과

선택한 쪽의 인덱스만
한 칸 움직입니다.

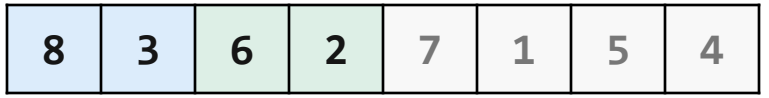
③ 병합 · 3번째 값 6 추가

왼쪽 8 > 오른쪽 6 → 오른쪽의 6을 붙입니다.
j는 1 → 2. 사용하지 않은 쪽의 인덱스는 그대로입니다.



파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

처음 배열에서 이번에 합치는 부분



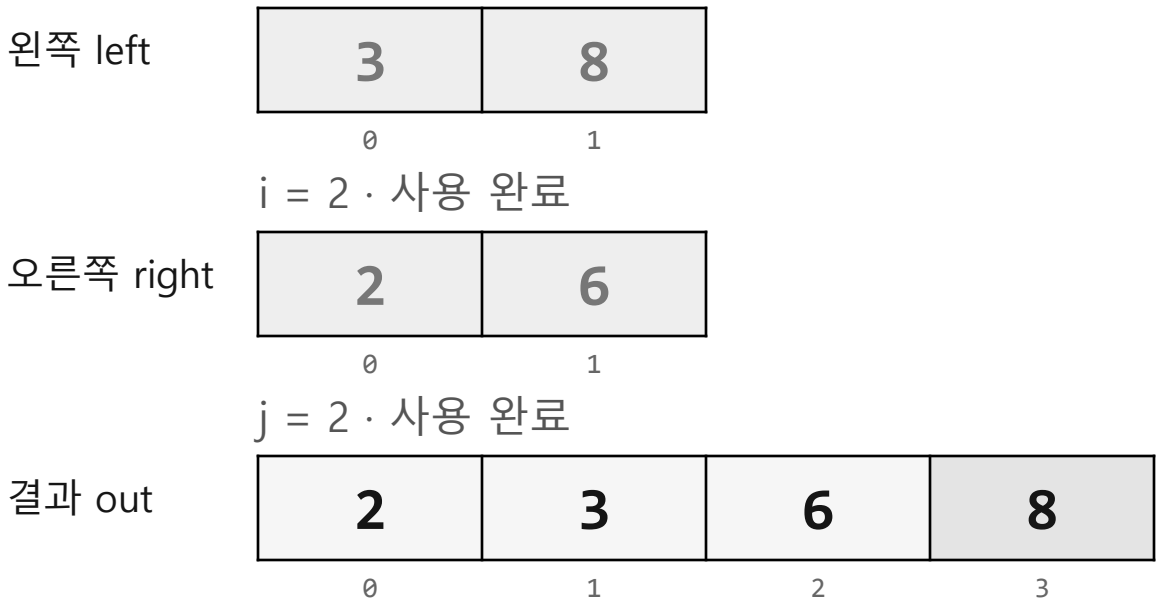
왼쪽 4개 전체

왼쪽: ①의 결과 오른쪽: ②의 결과

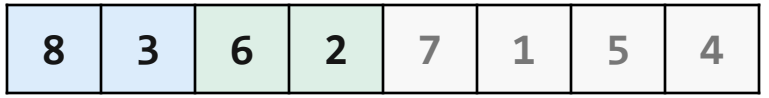
선택한 쪽의 인덱스만
한 칸 움직입니다.

③ 병합 · 4번째 값 8 추가

오른쪽을 다 사용했습니다. 남은 왼쪽의 8을 붙입니다.
i는 1 → 2. 이번 병합의 정렬 결과가 완성되었습니다.



처음 배열에서 이번에 합치는 부분



왼쪽 4개 전체

이번 병합의 정렬 결과

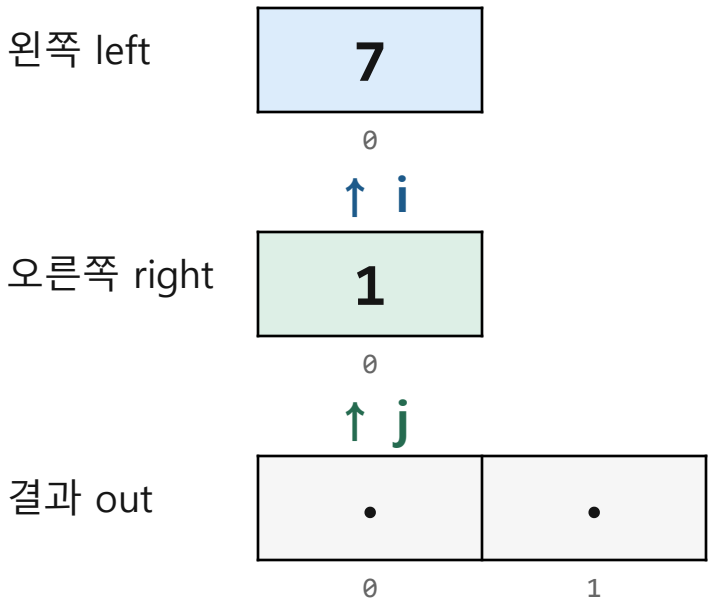
[2, 3, 6, 8]

왼쪽 4개 완료 → 오른쪽으로

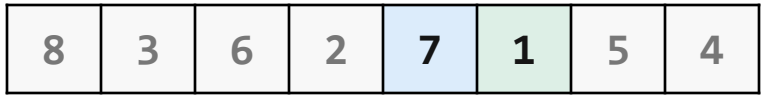
파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

④ [7]과 [1] 병합 · 시작

i = 0, j = 0. 왼쪽 7과 오른쪽 1을 비교합니다.
정렬된 두 결과를 받아 작은 값부터 새 결과에 넣습니다.



처음 배열에서 이번에 합치는 부분



오른쪽 4개의 앞쪽 두 개

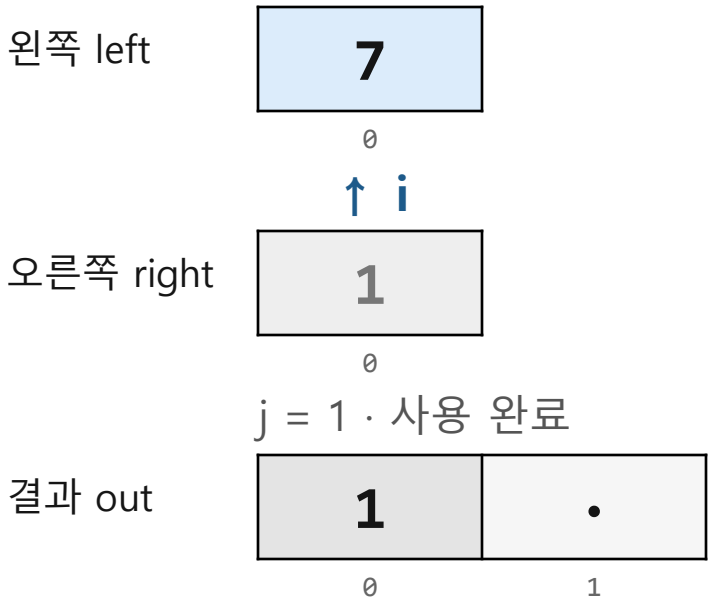
왼쪽 4개 [2,3,6,8]은 정렬 완료

선택한 쪽의 인덱스만
한 칸 움직입니다.

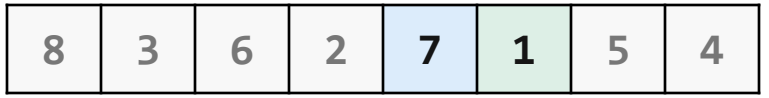
파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

④ 병합 · 1번째 값 1 추가

왼쪽 7 > 오른쪽 1 → 오른쪽의 1을 붙입니다.
j는 0 → 1. 사용하지 않은 쪽의 인덱스는 그대로입니다.



처음 배열에서 이번에 합치는 부분



오른쪽 4개의 앞쪽 두 개

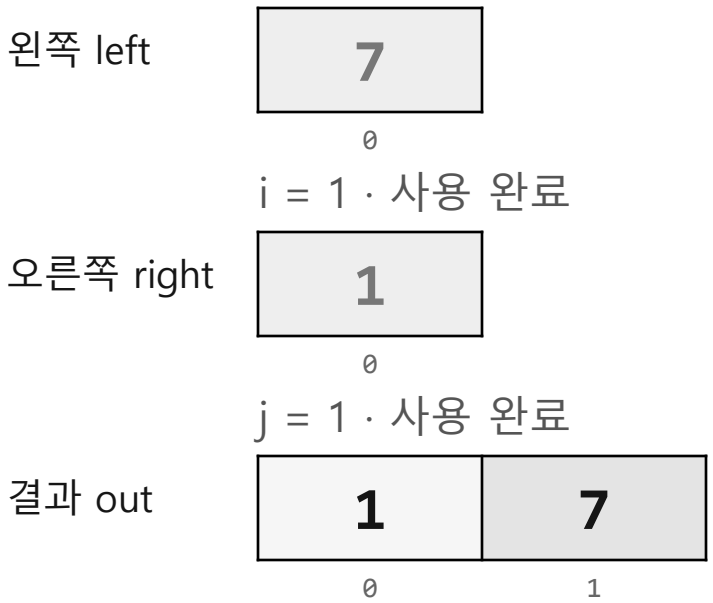
왼쪽 4개 [2,3,6,8]은 정렬 완료

선택한 쪽의 인덱스만
한 칸 움직입니다.

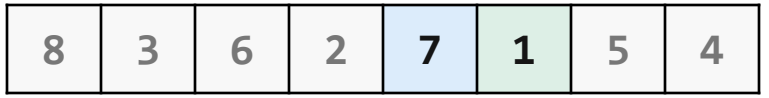
파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

④ 병합 · 2번째 값 7 추가

오른쪽을 다 사용했습니다. 남은 왼쪽의 7을 붙입니다.
i는 0 → 1. 이번 병합의 정렬 결과가 완성되었습니다.



처음 배열에서 이번에 합치는 부분



오른쪽 4개의 앞쪽 두 개

이번 병합의 정렬 결과

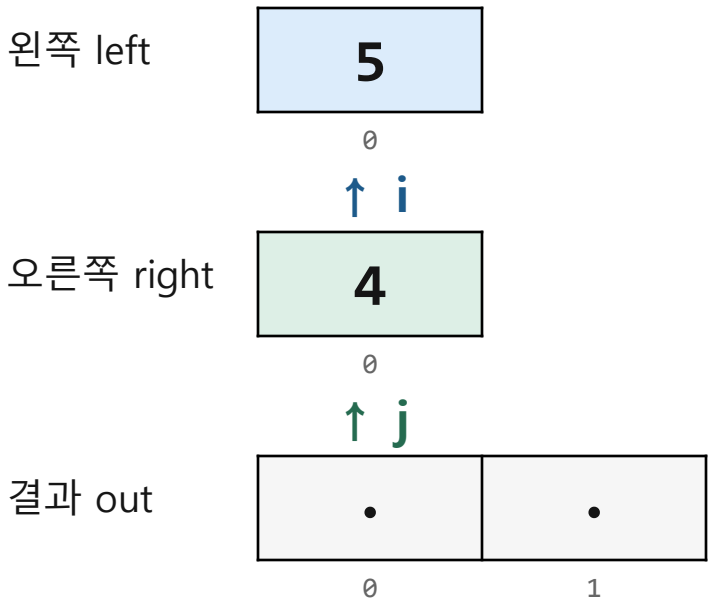
[1, 7]

⑥에서 왼쪽 입력으로 사용

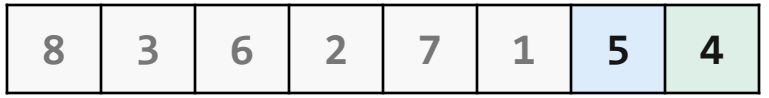
파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

⑤ [5]과 [4] 병합 · 시작

i = 0, j = 0. 왼쪽 5과 오른쪽 4을 비교합니다.
정렬된 두 결과를 받아 작은 값부터 새 결과에 넣습니다.



처음 배열에서 이번에 합치는 부분



오른쪽 4개의 뒤쪽 두 개

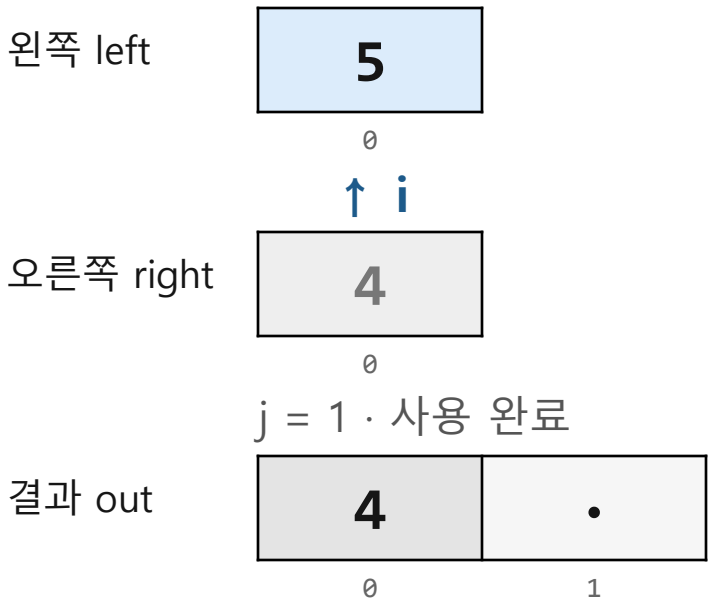
④의 [1,7]은 준비된 채 기다립니다.

선택한 쪽의 인덱스만
한 칸 움직입니다.

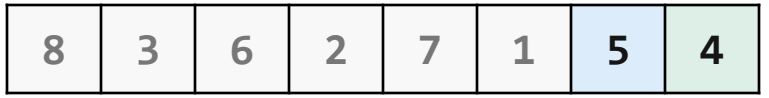
파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

⑤ 병합 · 1번째 값 4 추가

왼쪽 5 > 오른쪽 4 → 오른쪽의 4을 붙입니다.
j는 0 → 1. 사용하지 않은 쪽의 인덱스는 그대로입니다.



처음 배열에서 이번에 합치는 부분



오른쪽 4개의 뒤쪽 두 개

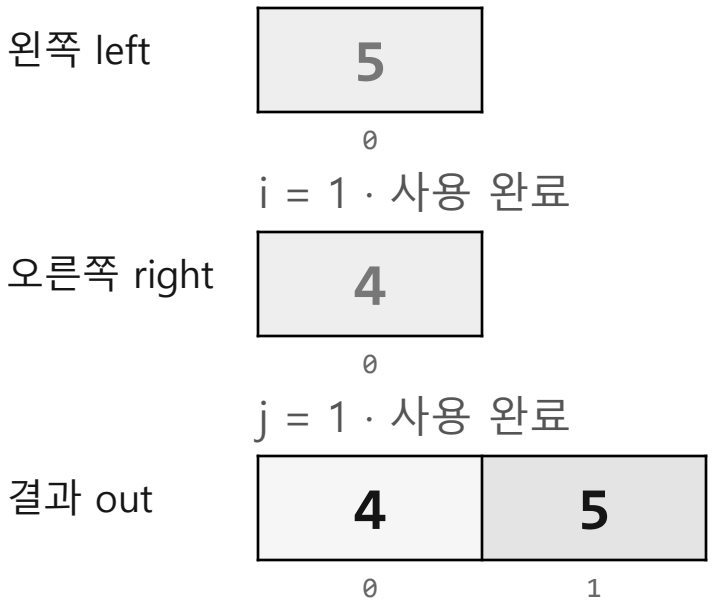
④의 [1,7]은 준비된 채 기다립니다.

선택한 쪽의 인덱스만
한 칸 움직입니다.

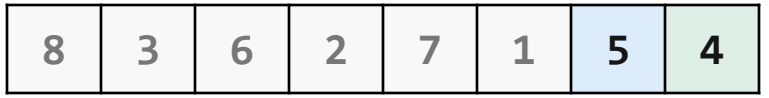
파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

⑤ 병합 · 2번째 값 5 추가

오른쪽을 다 사용했습니다. 남은 왼쪽의 5을 붙입니다.
i는 0 → 1. 이번 병합의 정렬 결과가 완성되었습니다.



처음 배열에서 이번에 합치는 부분



오른쪽 4개의 뒤쪽 두 개

이번 병합의 정렬 결과

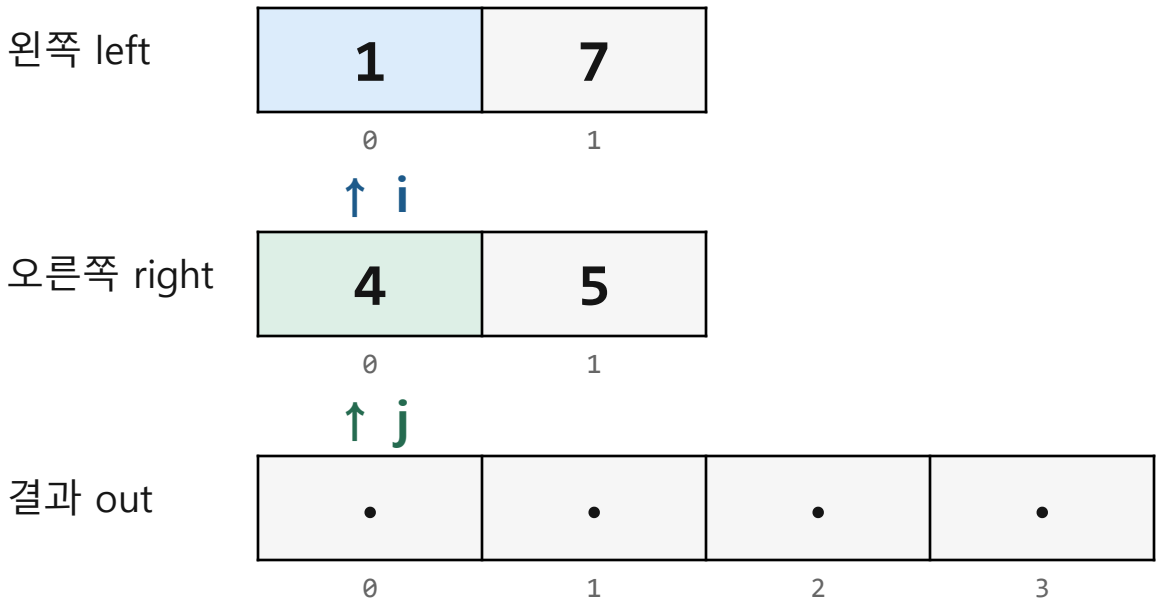
[4, 5]

⑥에서 오른쪽 입력으로 사용

파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

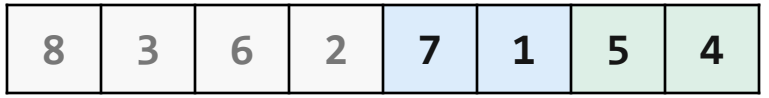
⑥ [1, 7]과 [4, 5] 병합 · 시작

i = 0, j = 0. 왼쪽 1과 오른쪽 4을 비교합니다.
정렬된 두 결과를 받아 작은 값부터 새 결과에 넣습니다.



파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

처음 배열에서 이번에 합치는 부분



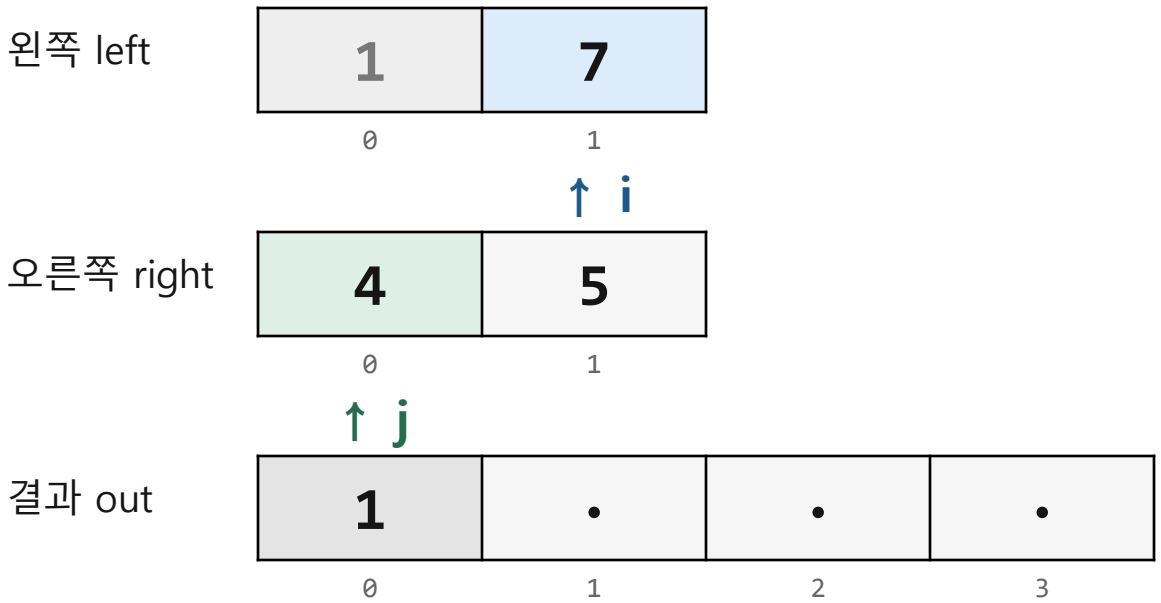
오른쪽 4개 전체

왼쪽: ④의 결과 오른쪽: ⑤의 결과

선택한 쪽의 인덱스만
한 칸 움직입니다.

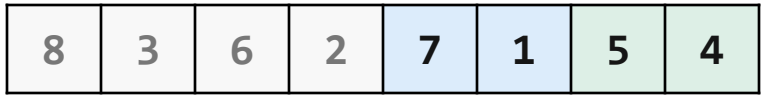
⑥ 병합 · 1번째 값 1 추가

왼쪽 1 ≤ 오른쪽 4 → 왼쪽의 1을 붙입니다.
i는 0 → 1. 사용하지 않은 쪽의 인덱스는 그대로입니다.



파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

처음 배열에서 이번에 합치는 부분



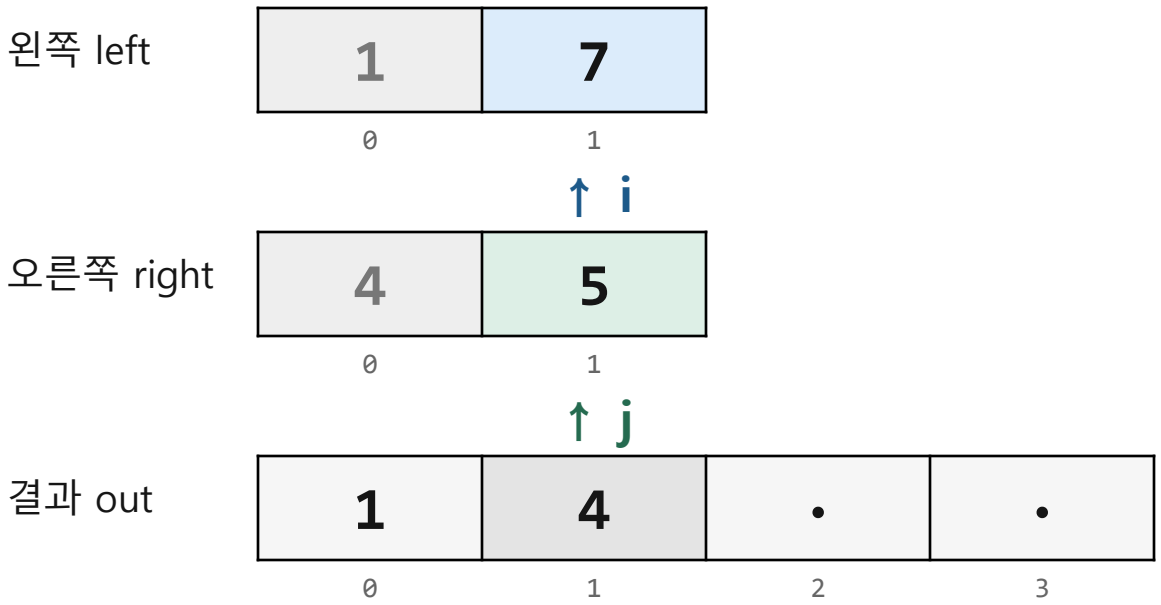
오른쪽 4개 전체

왼쪽: ④의 결과 오른쪽: ⑤의 결과

선택한 쪽의 인덱스만
한 칸 움직입니다.

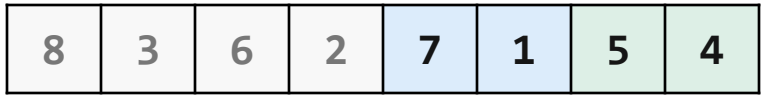
⑥ 병합 · 2번째 값 4 추가

왼쪽 7 > 오른쪽 4 → 오른쪽의 4을 붙입니다.
j는 0 → 1. 사용하지 않은 쪽의 인덱스는 그대로입니다.



파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

처음 배열에서 이번에 합치는 부분



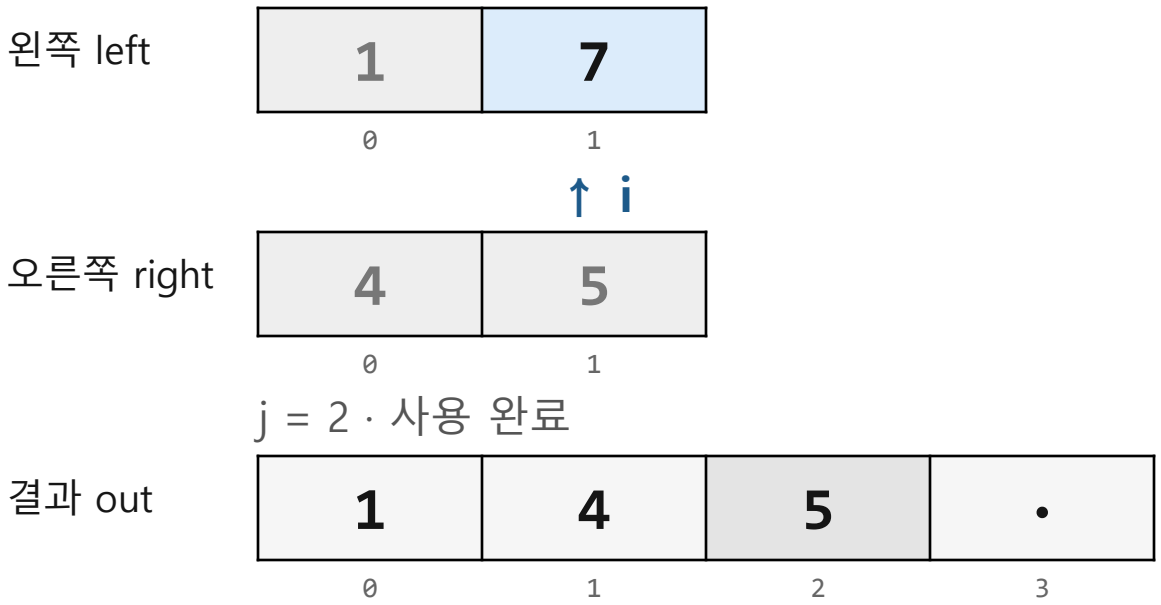
오른쪽 4개 전체

왼쪽: ④의 결과 오른쪽: ⑤의 결과

선택한 쪽의 인덱스만
한 칸 움직입니다.

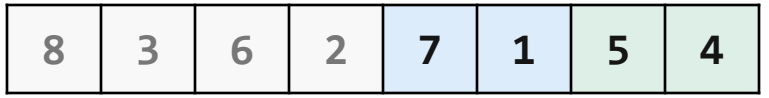
⑥ 병합 · 3번째 값 5 추가

왼쪽 7 > 오른쪽 5 → 오른쪽의 5을 붙입니다.
j는 1 → 2. 사용하지 않은 쪽의 인덱스는 그대로입니다.



파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

처음 배열에서 이번에 합치는 부분



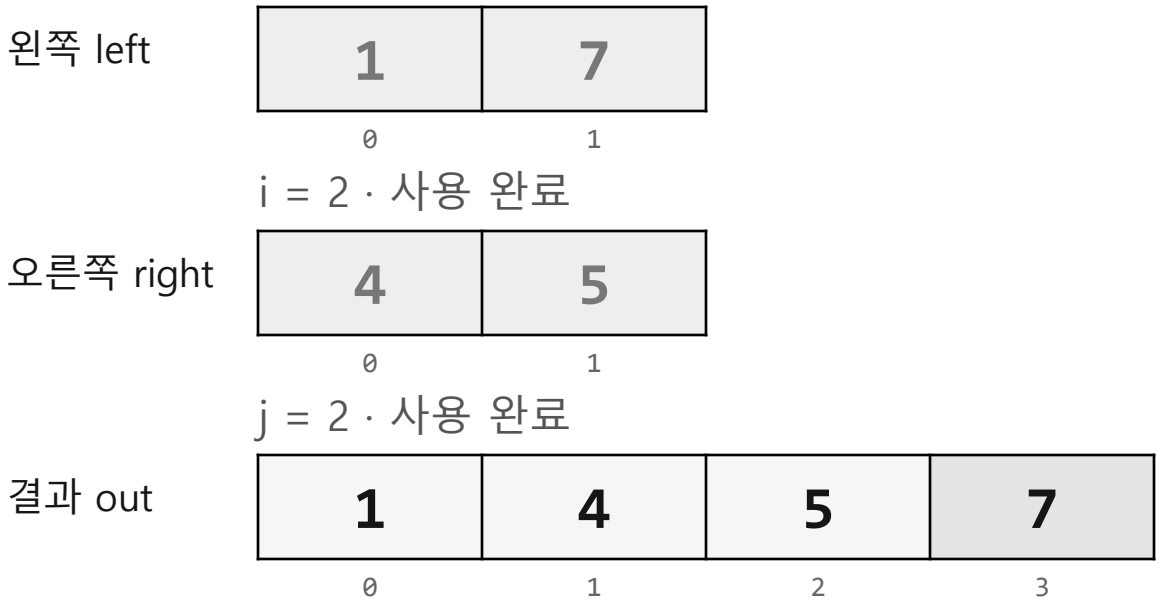
오른쪽 4개 전체

왼쪽: ④의 결과 오른쪽: ⑤의 결과

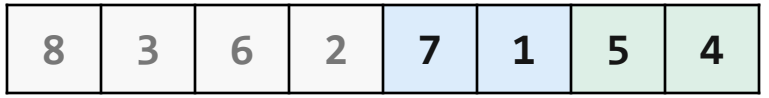
선택한 쪽의 인덱스만
한 칸 움직입니다.

⑥ 병합 · 4번째 값 7 추가

오른쪽을 다 사용했습니다. 남은 왼쪽의 7을 붙입니다.
i는 1 → 2. 이번 병합의 정렬 결과가 완성되었습니다.



처음 배열에서 이번에 합치는 부분



오른쪽 4개 전체

이번 병합의 정렬 결과

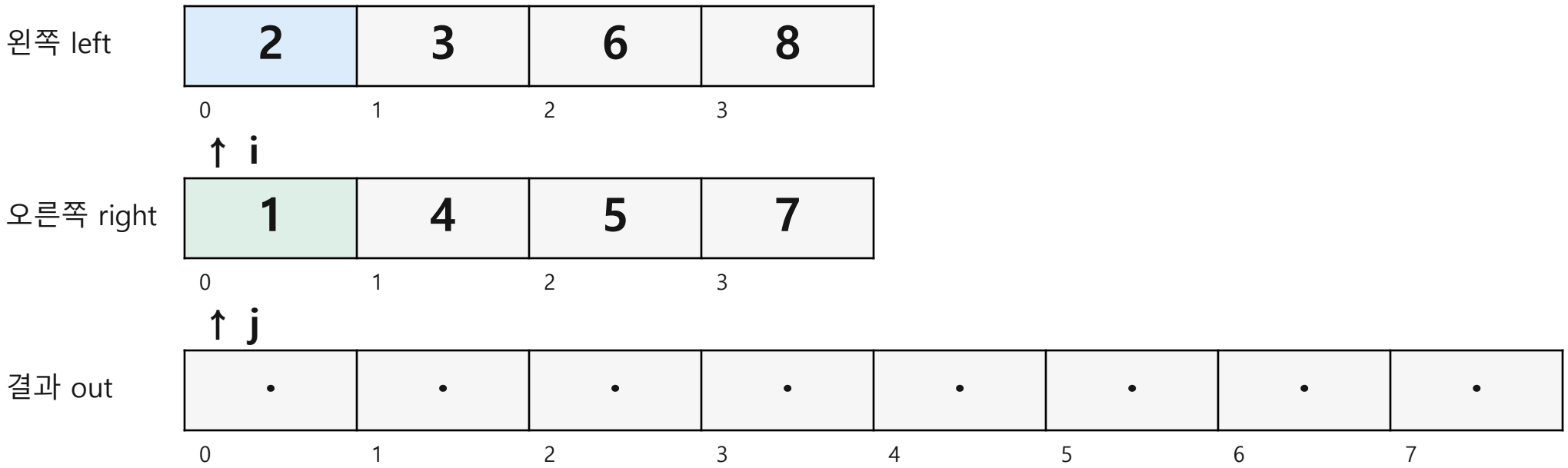
[1, 4, 5, 7]

⑦에서 오른쪽 입력으로 사용

파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

병합 시작 · 두 화살표가 첫 칸

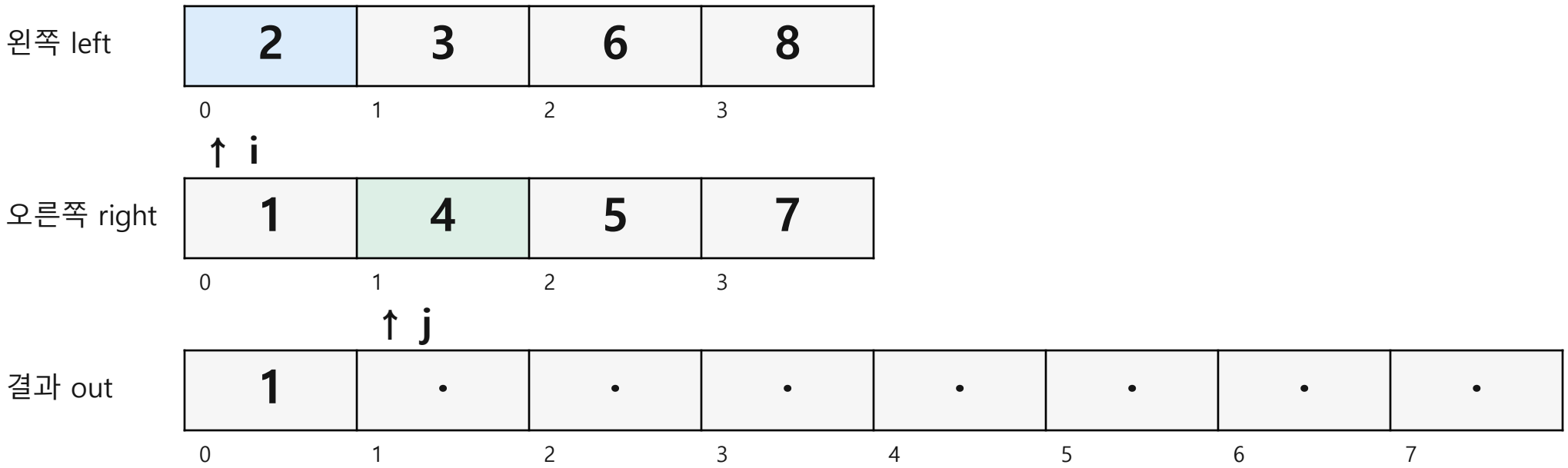
i = 0, j = 0입니다. 왼쪽 2와 오른쪽 1을 비교합니다.
결과 배열은 아직 비어 있습니다.



한 페이지씩 넘기며 화살표 위치와 바뀐 값을 비교하세요.

병합 1단계 · 1을 붙인 뒤

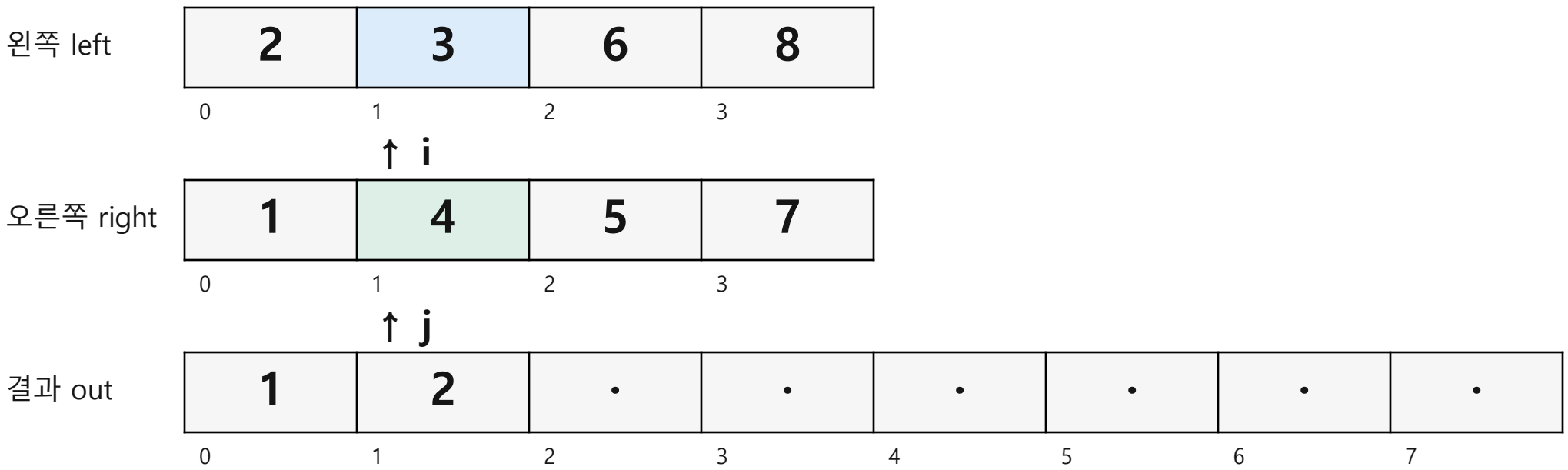
왼쪽 2 > 오른쪽 1. 오른쪽 값을 결과 끝에 붙입니다.
j를 0에서 1로 옮깁니다. i = 0는 그대로입니다.



한 페이지씩 넘기며 화살표 위치와 바뀐 값을 비교하세요.

병합 2단계 · 2을 붙인 뒤

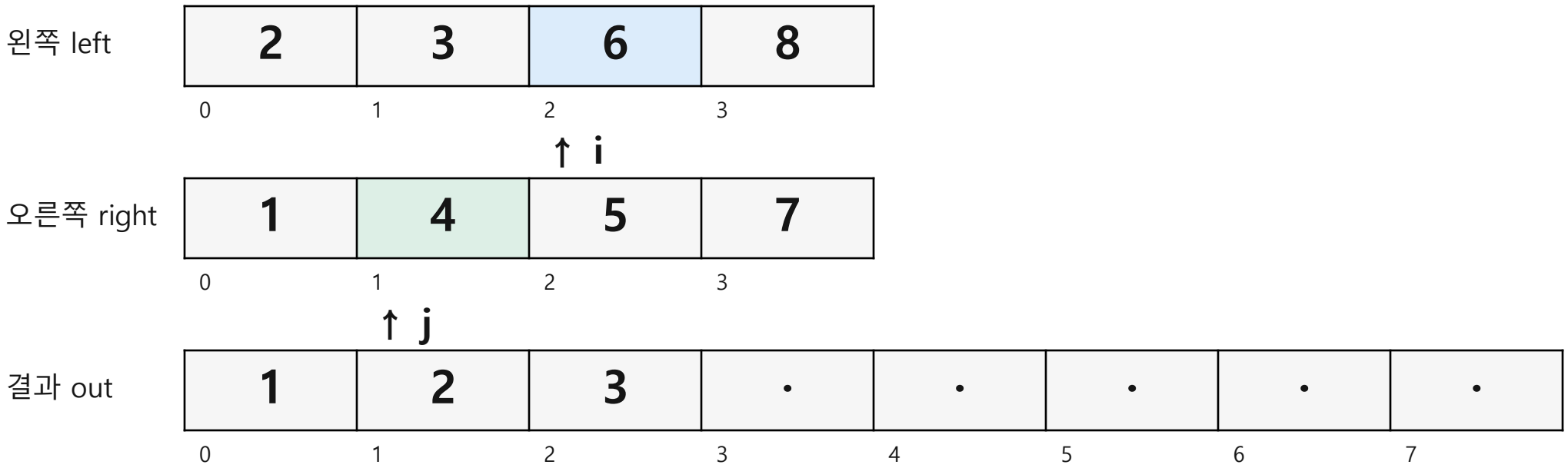
왼쪽 2 ≤ 오른쪽 4. 왼쪽 값을 결과 끝에 붙입니다.
i를 0에서 1로 옮깁니다. j = 1는 그대로입니다.



한 페이지씩 넘기며 화살표 위치와 바뀐 값을 비교하세요.

병합 3단계 · 3을 붙인 뒤

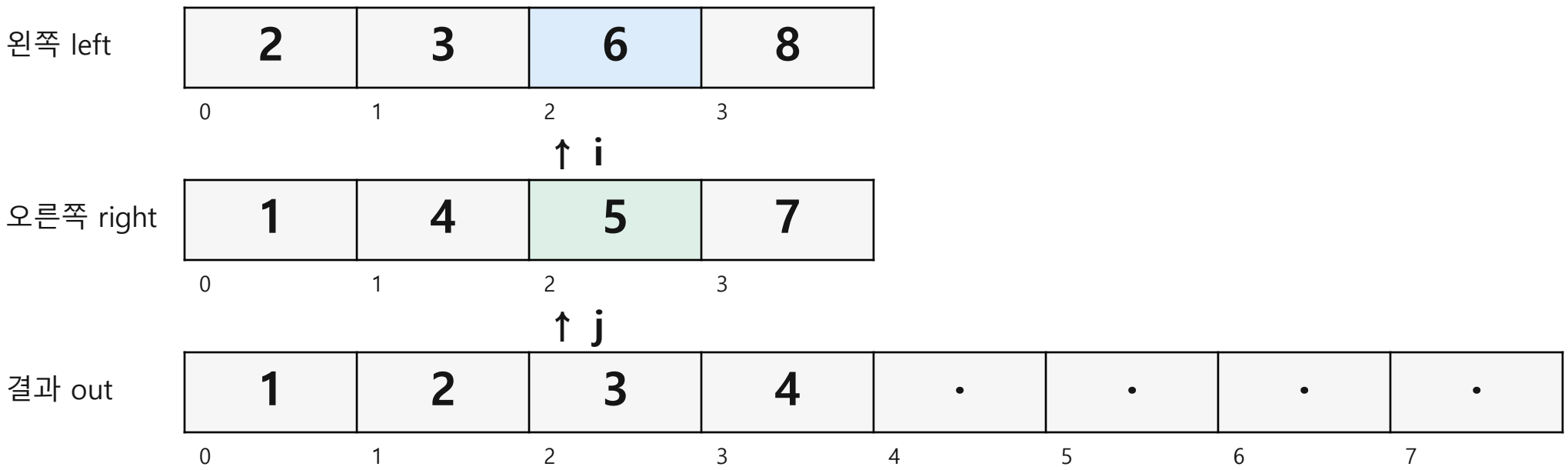
왼쪽 3 ≤ 오른쪽 4. 왼쪽 값을 결과 끝에 붙입니다.
i를 1에서 2로 옮깁니다. j = 1는 그대로입니다.



한 페이지씩 넘기며 화살표 위치와 바뀐 값을 비교하세요.

병합 4단계 · 4을 붙인 뒤

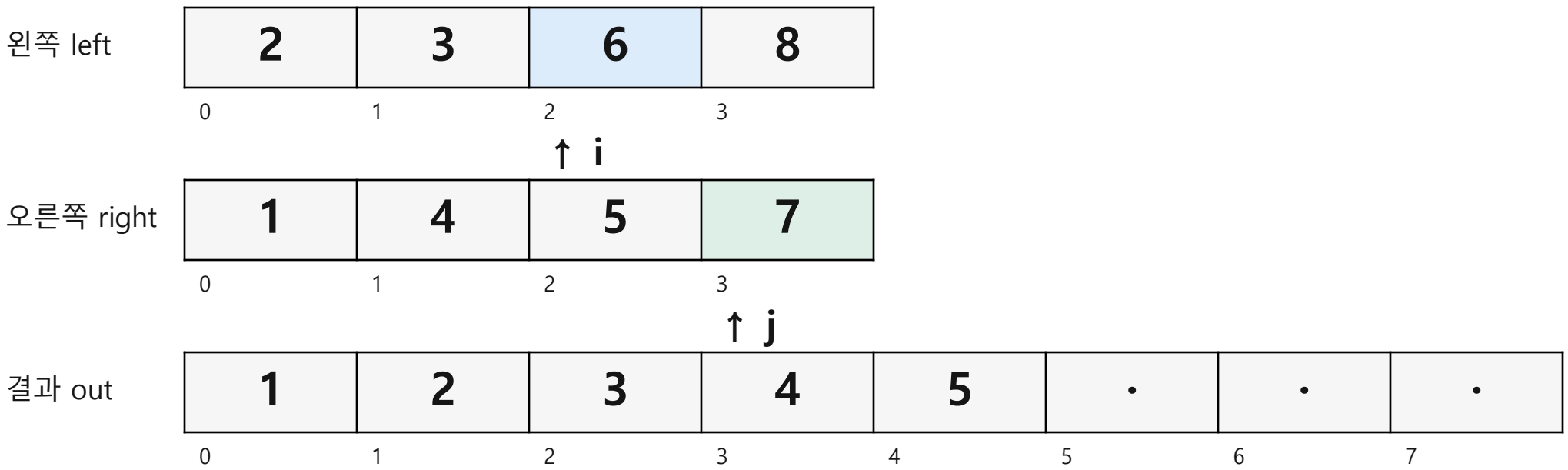
왼쪽 6 > 오른쪽 4. 오른쪽 값을 결과 끝에 붙입니다.
j를 1에서 2로 옮깁니다. i = 2는 그대로입니다.



한 페이지씩 넘기며 화살표 위치와 바뀐 값을 비교하세요.

병합 5단계 · 5을 붙인 뒤

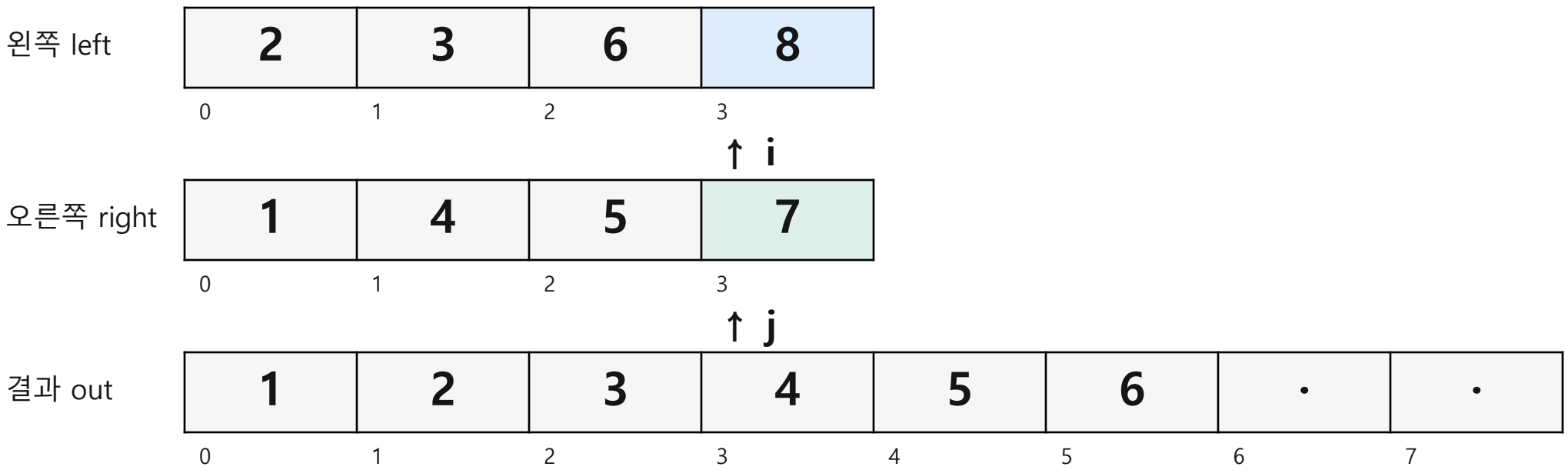
왼쪽 6 > 오른쪽 5. 오른쪽 값을 결과 끝에 붙입니다.
j를 2에서 3로 옮깁니다. i = 2는 그대로입니다.



한 페이지씩 넘기며 화살표 위치와 바뀐 값을 비교하세요.

병합 6단계 · 6을 붙인 뒤

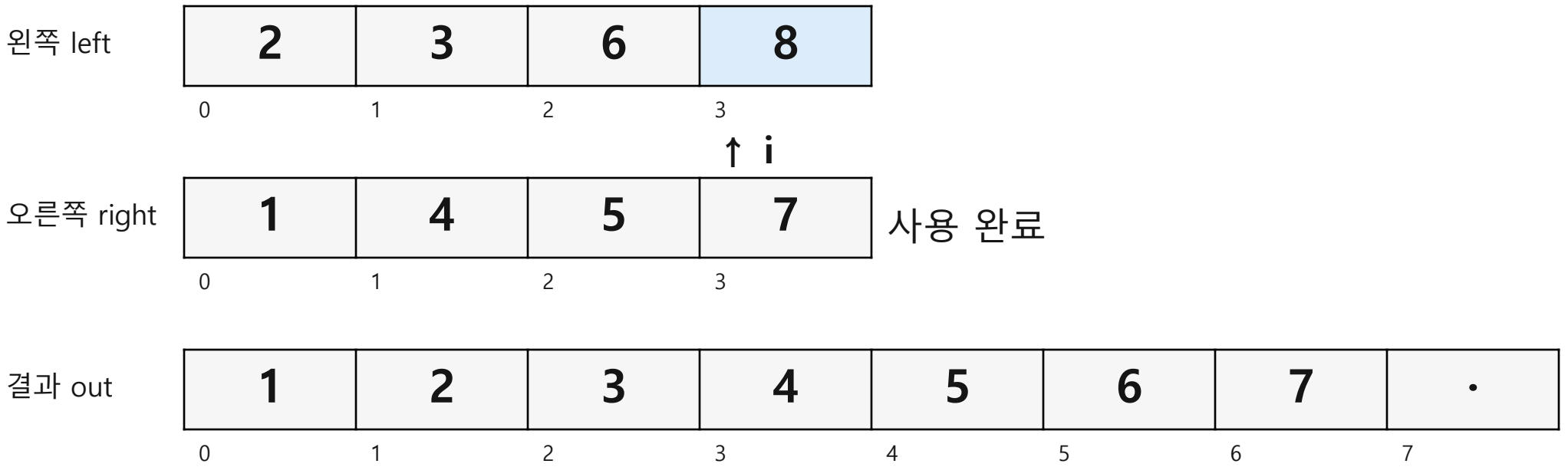
왼쪽 6 ≤ 오른쪽 7. 왼쪽 값을 결과 끝에 붙입니다.
i를 2에서 3로 옮깁니다. j = 3는 그대로입니다.



한 페이지씩 넘기며 화살표 위치와 바뀐 값을 비교하세요.

병합 7단계 · 7을 붙인 뒤

왼쪽 8 > 오른쪽 7. 오른쪽 값을 결과 끝에 붙입니다.
j를 3에서 4로 옮깁니다. i = 3는 그대로입니다.



한 페이지씩 넘기며 화살표 위치와 바뀐 값을 비교하세요.

병합 8단계 · 8을 붙인 뒤

오른쪽을 다 사용했습니다. 왼쪽의 8을 결과 끝에 붙입니다.



한 페이지씩 넘기며 화살표 위치와 바뀐 값을 비교하세요.

Python: 병합 함수

```
def merge(left, right):
    out = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            out.append(left[i])
            i += 1
        else:
            out.append(right[j])
            j += 1
    while i < len(left):
        out.append(left[i])
        i += 1
    while j < len(right):
        out.append(right[j])
        j += 1
    return out
```

Python: 재귀 호출 순서

```
def merge_sort(a):
    if len(a) <= 1:
        return a[:]
    mid = len(a) // 2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
    return merge(left, right)

a = [8, 3, 6, 2, 7, 1, 5, 4]
print(merge_sort(a))
```

C: 정렬된 두 구간의 병합

```
void merge(int a[], int tmp[], int lo, int mid, int hi) {
    int i = lo, j = mid + 1, k = lo;
    while (i <= mid && j <= hi) {
        if (a[i] <= a[j]) tmp[k++] = a[i++];
        else tmp[k++] = a[j++];
    }
    while (i <= mid) tmp[k++] = a[i++];
    while (j <= hi) tmp[k++] = a[j++];
    for (int t = lo; t <= hi; t++) a[t] = tmp[t];
}
```

C: 재귀 호출과 실행

```
/* 앞 슬라이드의 merge 함수를 함께 넣는다. */
void merge_sort(int a[], int tmp[], int lo, int hi) {
    if (lo >= hi) return;
    int mid = lo + (hi - lo) / 2;
    merge_sort(a, tmp, lo, mid);
    merge_sort(a, tmp, mid + 1, hi);
    merge(a, tmp, lo, mid, hi);
}
/* 실행 예: main 안에서 */
int a[] = {8, 3, 6, 2, 7, 1, 5, 4};
int tmp[8];
merge_sort(a, tmp, 0, 7);
/* a는 {1, 2, 3, 4, 5, 6, 7, 8}이 된다. */
```

왜 Θ(n log n)인가?

한 단계의 병합에서 모든 원소를 합쳐 n개 처리한다.
절반으로 나누므로 단계 수는 Θ(log n)이다.
최선·평균·최악 모두 Θ(n log n)
같은 값이면 왼쪽을 먼저 선택하면 안정 정렬이다.

단계별 처리량

2개 × 4쌍 = 8개	**4개 × 2쌍 = 8개**	**8개 × 1쌍 = 8개**

병합 정렬 확인

왼쪽 [2, 5], 오른쪽 [1, 3, 7]을 병합하자.
① 첫 번째로 선택하는 값과 움직이는 포인터는?
② 왼쪽이 먼저 끝났을 때 어떤 값을 붙이는가?
③ 재귀 코드에서 merge를 두 재귀 호출 뒤에 쓰는 이유는?

왼쪽

2	5

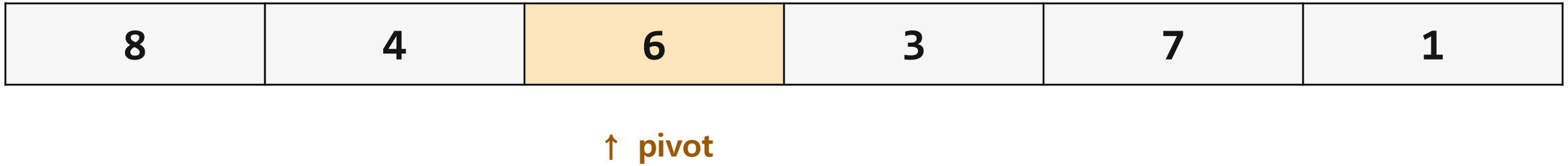
오른쪽

1	3	7

퀵 정렬 (Quick Sort)

피벗은 원소들을 나누는 비교 기준값입니다.
이번 구현은 피벗 이하를 왼쪽, 피벗 초과를 오른쪽에 모읍니다.

이번 입력



분할 뒤 피벗의 자리를 정하고, 피벗 양쪽 구간을 다시 정렬합니다.

피벗 선택: 가운데 칸의 6

현재 구간은 lo = 0부터 hi = 5까지입니다.
가운데 인덱스 (lo + hi) // 2 = 2에서 피벗 a[2] = 6을 고릅니다.



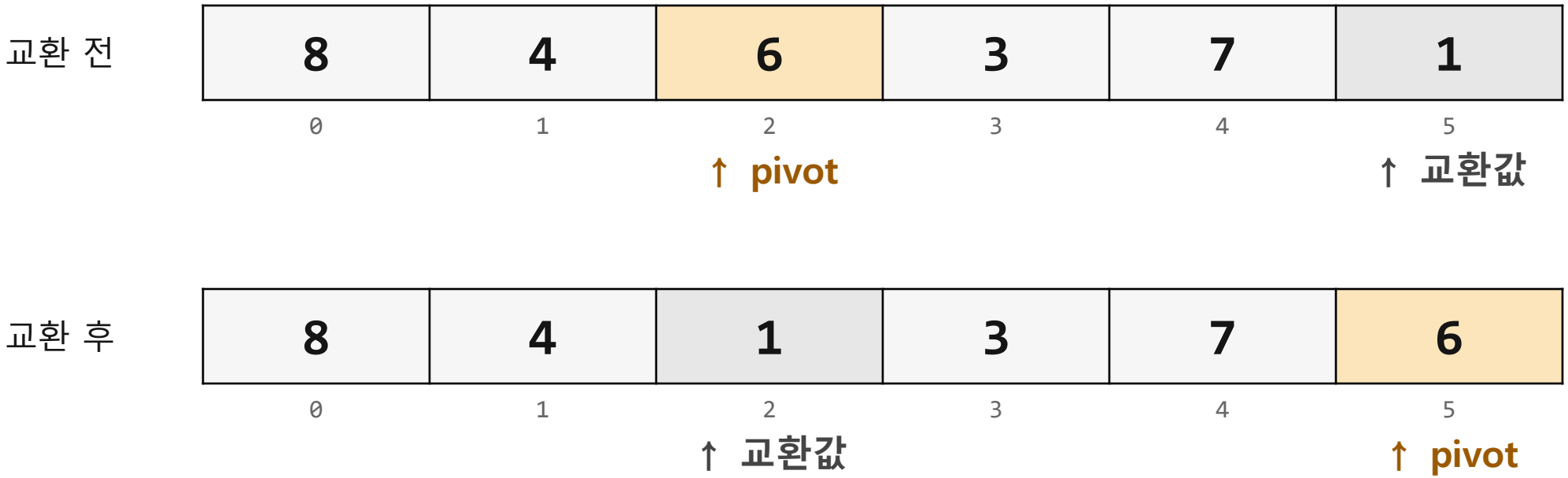

```
pivot_idx = (lo + hi) // 2
```

이번 코드가 사용하는 피벗 선택 규칙입니다.

피벗 6과 마지막 값 1의 교환

피벗 a[2] = 6과 마지막 값 a[5] = 1을 교환합니다.
6은 인덱스 5로, 1은 인덱스 2로 이동합니다.

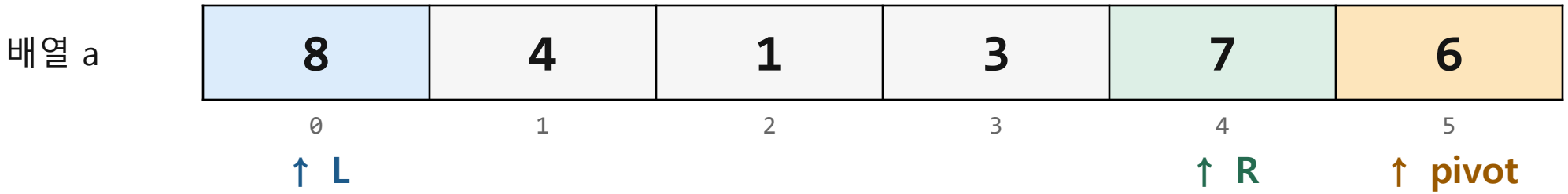
```
a[pivot_idx], a[hi] = a[hi], a[pivot_idx]
```



끝의 6은 검사 동안 임시로 두는 값입니다. 최종 위치는 분할이 끝난 뒤 정합니다.

① 탐색 시작: L = 0, R = 4

L = 0, R = 4에서 시작합니다.
마지막 칸의 피벗 6을 제외하고, 인덱스 0~4를 검사합니다.
첫 비교에서 8 > 6이므로 L은 인덱스 0에서 멈춥니다.

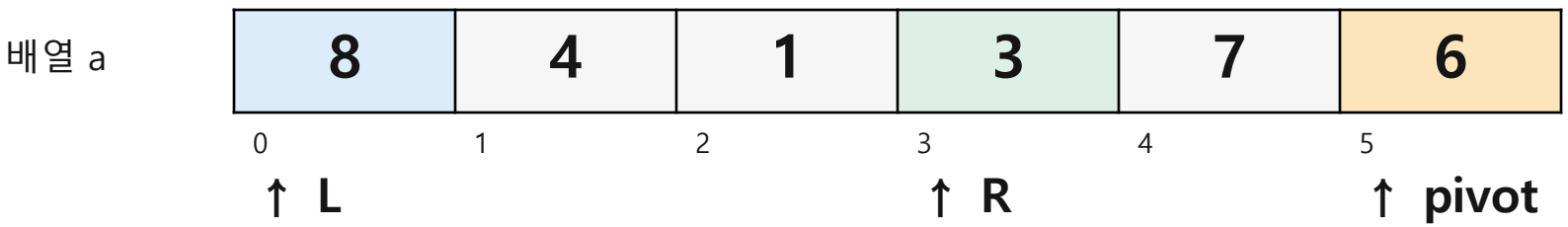



```
pivot = a[hi]
L, R = lo, hi - 1
```

다음: R이 7을 지나, 왼쪽으로 보낼 값 3을 찾습니다.

② R이 4에서 3으로 이동

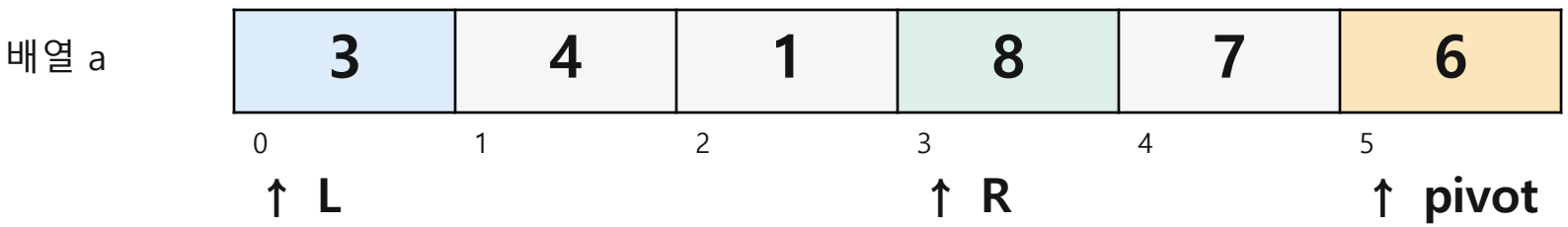
a[4] = 7 > 6이므로 R을 왼쪽으로 한 칸 옮깁니다.
a[3] = 3 ≤ 6이므로 여기서 멈춥니다. L은 0에 그대로 있습니다.



한 페이지씩 넘기며 화살표 위치와 바뀐 값을 비교하세요.

③ 두 포인터가 가리킨 8과 3 교환

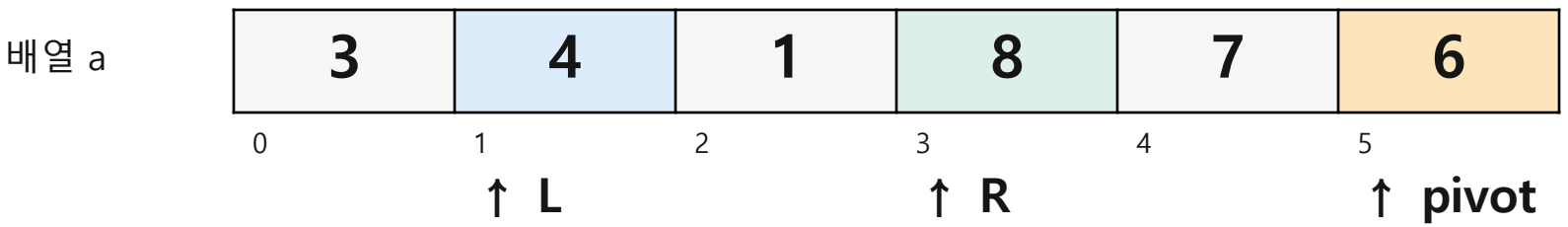
L = 0, R = 3이고 아직 L ≤ R입니다.
a[0]과 a[3]을 교환합니다. 포인터 위치는 그대로입니다.



한 페이지씩 넘기며 화살표 위치와 바뀐 값을 비교하세요.

④ L이 0에서 1로 이동

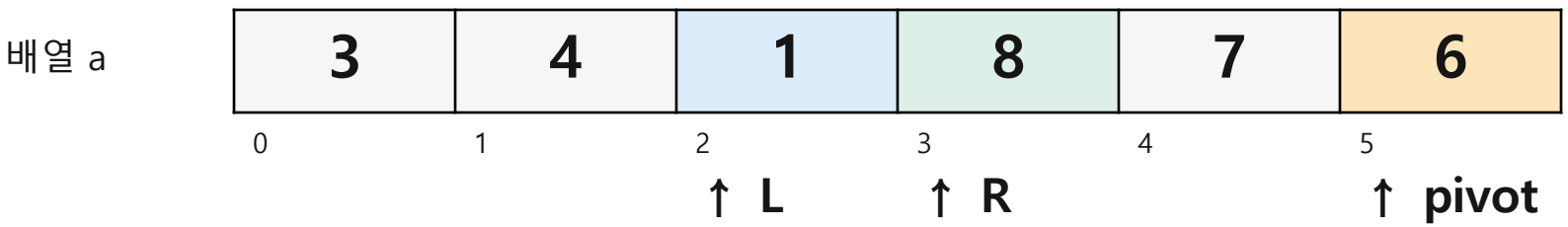
a[0] = 3 ≤ 6이므로 L을 오른쪽으로 한 칸 옮깁니다.
R은 인덱스 3에 그대로 있습니다.



한 페이지씩 넘기며 화살표 위치와 바뀐 값을 비교하세요.

⑤ L이 1에서 2로 이동

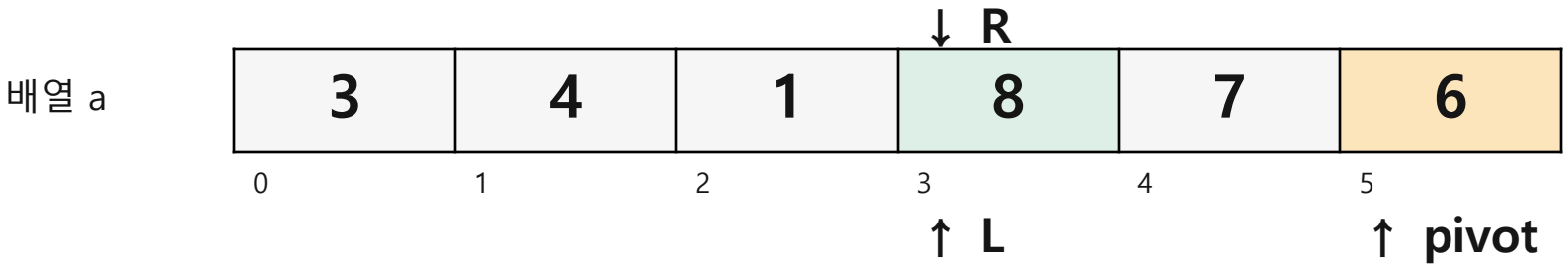
a[1] = 4 ≤ 6이므로 L을 다시 한 칸 옮깁니다.
지금 L이 가리키는 값은 a[2] = 1입니다.



한 페이지씩 넘기며 화살표 위치와 바뀐 값을 비교하세요.

⑥ L이 2에서 3으로 이동

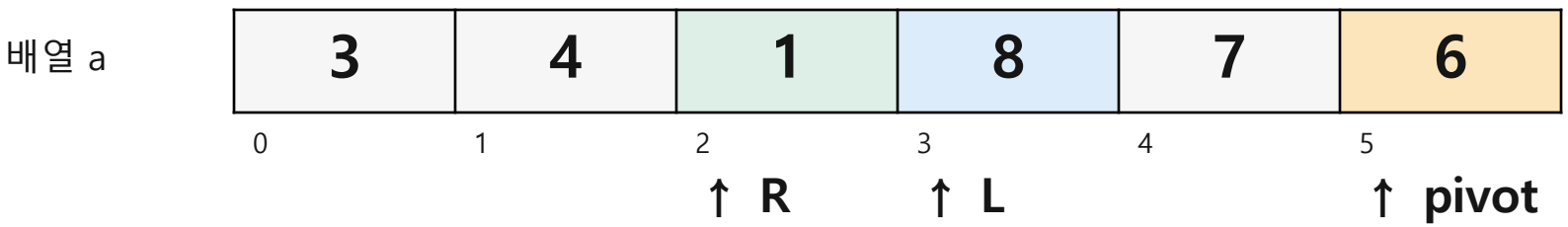
a[2] = 1 ≤ 6이므로 L을 한 칸 옮깁니다.
a[3] = 8 > 6이므로 L의 이동은 여기서 끝납니다.



한 페이지씩 넘기며 화살표 위치와 바뀐 값을 비교하세요.

⑦ R이 3에서 2로 이동 · 교차

a[3] = 8 > 6이므로 R을 왼쪽으로 한 칸 옮깁니다.
L = 3 > R = 2가 되었으므로 두 값의 교환을 끝냅니다.

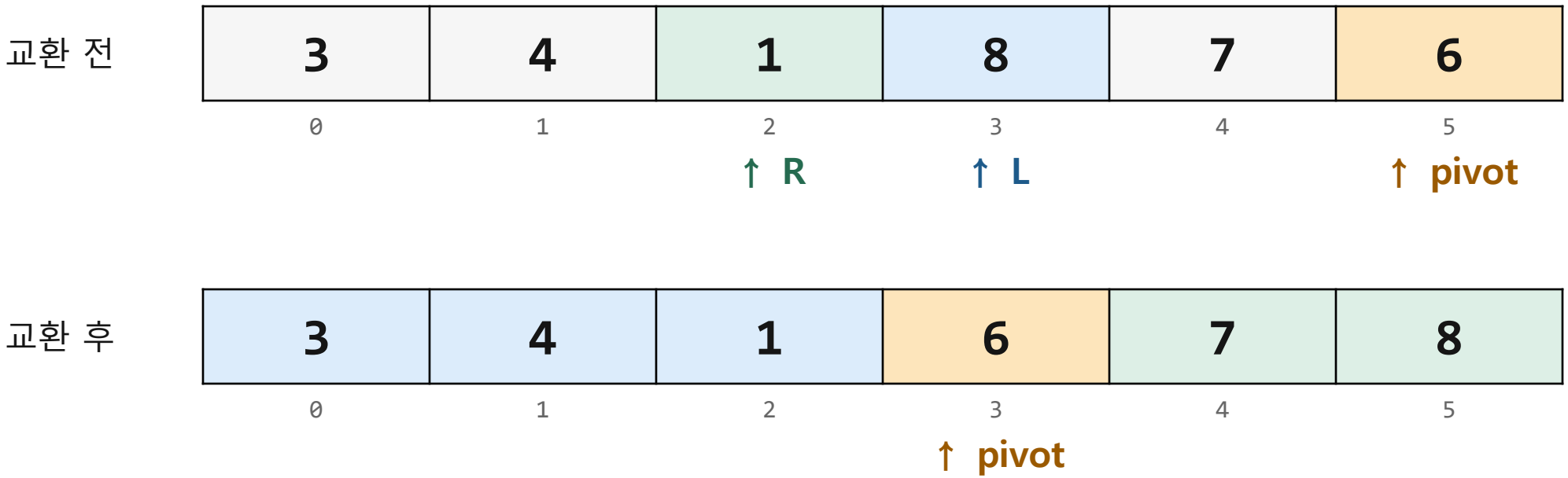


한 페이지씩 넘기며 화살표 위치와 바뀐 값을 비교하세요.

⑧ 피벗을 두 구간 사이에 놓는 교환

L = 3, R = 2로 교차했습니다. L은 큰 값 구간의 첫 칸입니다.
그 칸의 8과 맨 끝 피벗 6을 교환합니다. 피벗의 최종 위치는 3입니다.

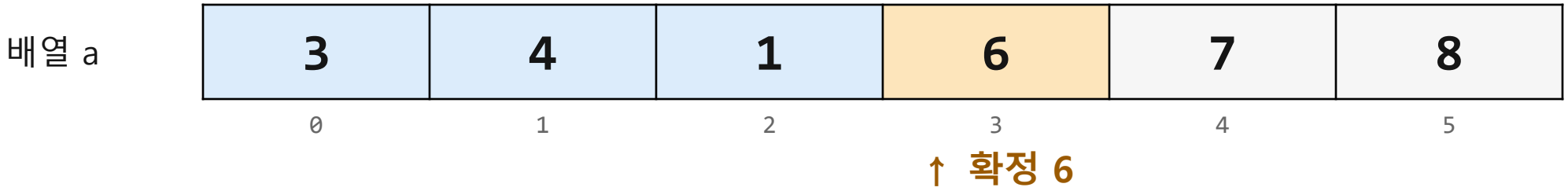
```
a[L], a[hi] = a[hi], a[L]
```



파랑: 6 이하인 왼쪽 구간 주황: 피벗 6 초록: 6보다 큰 오른쪽 구간

다음 호출: 왼쪽 구간 0~2부터

피벗 6의 위치 p = 3은 확정됐습니다. 다음 정렬에서 제외합니다.
왼쪽 [3,4,1]을 먼저 정렬하고, 그 호출이 끝나면 오른쪽 [7,8]을 정렬합니다.




```
quick_sort(a, 0, 2)
quick_sort(a, 4, 5)
```

먼저 왼쪽 호출을 끝냅니다. 오른쪽 호출은 그동안 기다립니다.

왼쪽 [3,4,1]: 피벗 4의 이동

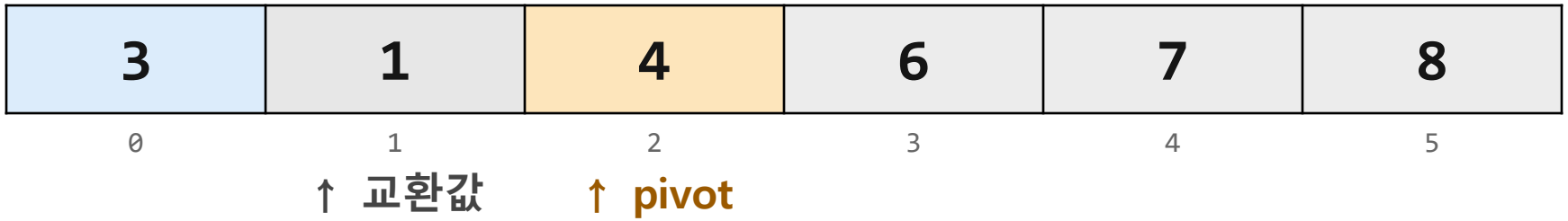
lo = 0, hi = 2이므로 가운데 인덱스는 1입니다. 피벗은 a[1] = 4입니다.
피벗 4와 이 구간의 마지막 값 a[2] = 1을 교환합니다.

```
a[1], a[2] = a[2], a[1]
```

교환 전



교환 후



현재 구간의 마지막 칸은 hi = 2입니다.

피벗 4: L = 0, R = 1에서 시작

검사할 값은 3과 1입니다. 피벗 4는 인덱스 2에 따로 둡니다.
L은 3을, R은 1을 가리킵니다. 먼저 L의 값부터 검사합니다.

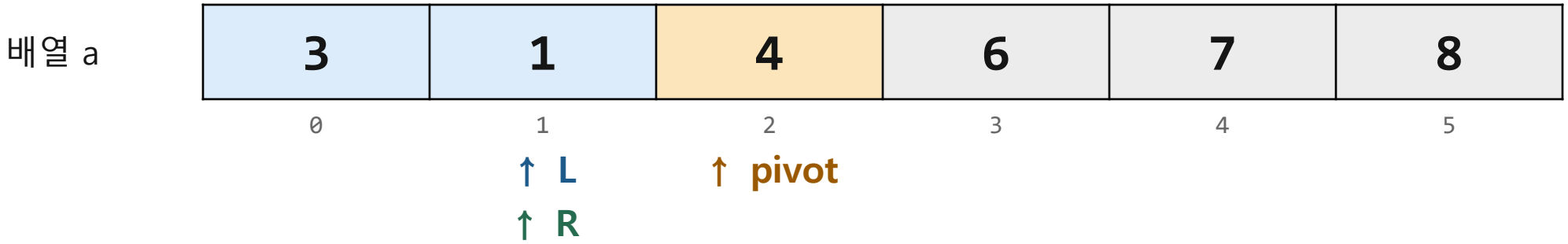



```
pivot = 4
L, R = 0, 1
```

3 ≤ 4이므로 다음에 L을 한 칸 늘립니다.

피벗 4: L이 0에서 1로 이동

a[0] = 3 ≤ 4이므로 L을 1로 늘립니다. 배열 값은 그대로입니다.
이제 L과 R은 모두 인덱스 1의 값 1을 가리킵니다.

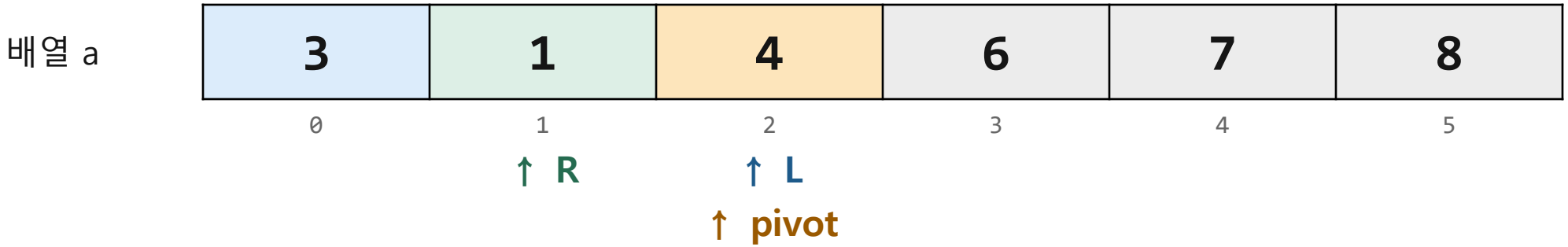


L = 1

L = R인 칸도 검사합니다. 1 ≤ 4이므로 L이 다시 이동합니다.

피벗 4: 교차 후 인덱스 2 확정

a[1] = 1 ≤ 4이므로 L = 2, R = 1로 교차합니다. 탐색을 끝냅니다.
L = hi = 2이므로 마지막 교환은 4와 자기 자신입니다. 배열은 그대로입니다.



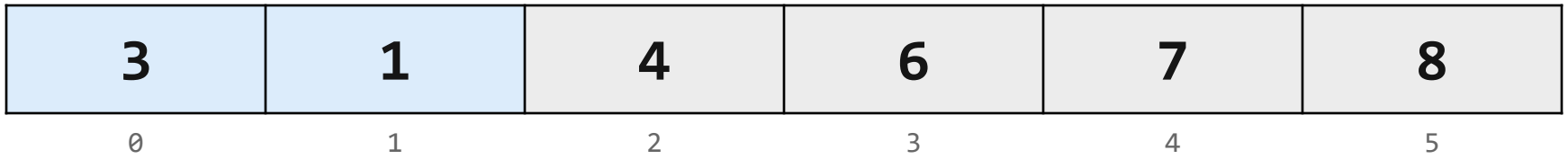

```
a[2], a[2] = a[2], a[2]
```

partition의 반환값 p = 2입니다. 앞의 [3,1]은 아직 정렬해야 합니다.

다음 왼쪽 호출: 구간 0~1

피벗 4의 위치 p = 2를 제외하고, 왼쪽 [3,1]을 다시 정렬합니다.
현재 lo = 0, hi = 1입니다. 4와 6의 칸은 이번 호출에서 움직이지 않습니다.

배열 a




```
quick_sort(a, 0, 1)
```

원소가 두 개이므로 한 번 더 분할합니다.

왼쪽 [3,1]: 피벗 3의 이동

가운데 인덱스 (0 + 1) // 2 = 0이므로 피벗은 a[0] = 3입니다.
피벗 3과 현재 구간의 마지막 값 a[1] = 1을 교환합니다.

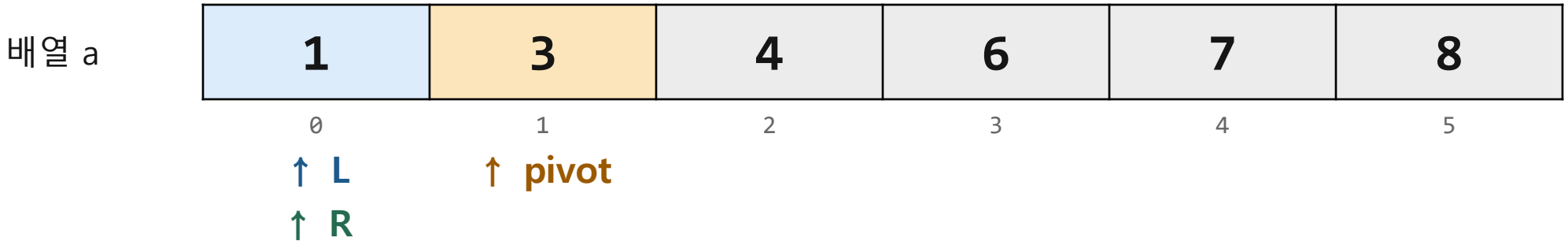
```
a[0], a[1] = a[1], a[0]
```



이번 구간의 hi는 1입니다. 그 뒤의 칸은 바꾸지 않습니다.

피벗 3: L = R = 0에서 시작

피벗 3은 인덱스 1에 두고, 앞의 값 1만 검사합니다.
L과 R은 모두 인덱스 0을 가리킵니다. 1 ≤ 3이므로 L을 늘립니다.

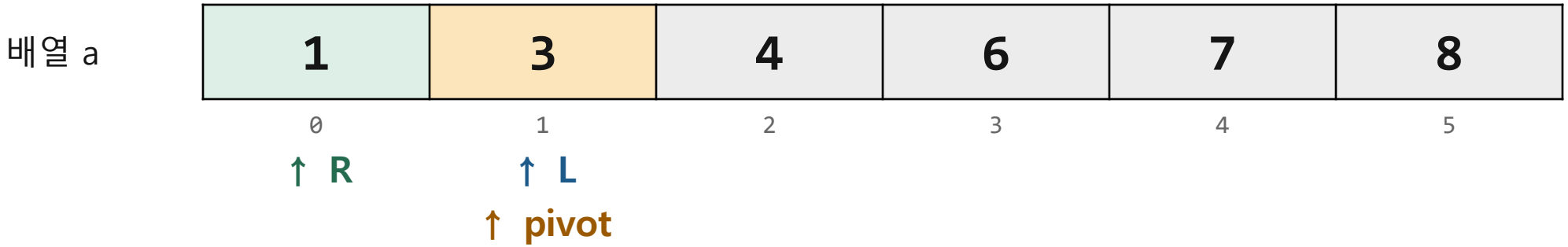



```
pivot = 3
L, R = 0, 0
```

L = R이어도 현재 칸을 한 번 검사합니다.

피벗 3: 교차 후 인덱스 1 확정

1 ≤ 3이므로 L = 1, R = 0이 되어 탐색을 끝냅니다.
L = hi = 1입니다. 마지막 교환은 3과 자기 자신이므로 배열은 그대로입니다.

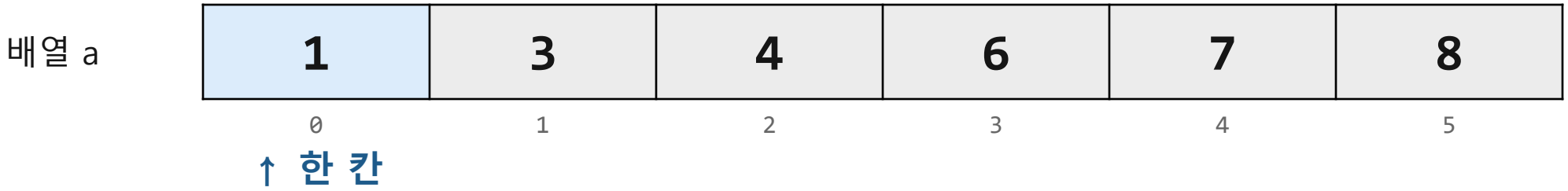



```
a[1], a[1] = a[1], a[1]
```

partition의 반환값 p = 1입니다. 다음 구간은 [0,0]과 [2,1]입니다.

종료 조건: 한 칸과 빈 구간

quick_sort(a, 0, 0)은 원소 하나입니다. quick_sort(a, 2, 1)은 빈 구간입니다.
두 호출 모두 lo ≥ hi이므로 partition을 호출하지 않고 바로 돌아옵니다.

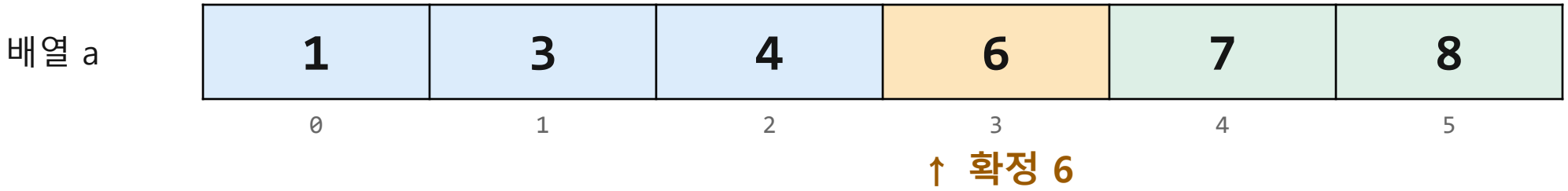



```
quick_sort(a, 0, 0)
quick_sort(a, 2, 1)
```

종료 조건 lo ≥ hi는 원소 하나인 경우와 빈 구간을 함께 처리합니다.

왼쪽 정렬 종료와 원래 호출로 복귀

구간 0~1을 끝내고 0~2 호출로 돌아옵니다. 오른쪽 [3,2]도 빈 구간입니다.
왼쪽 [1,3,4]를 모두 끝냈으므로, 원래 0~5 호출에서 오른쪽 4~5를 시작합니다.



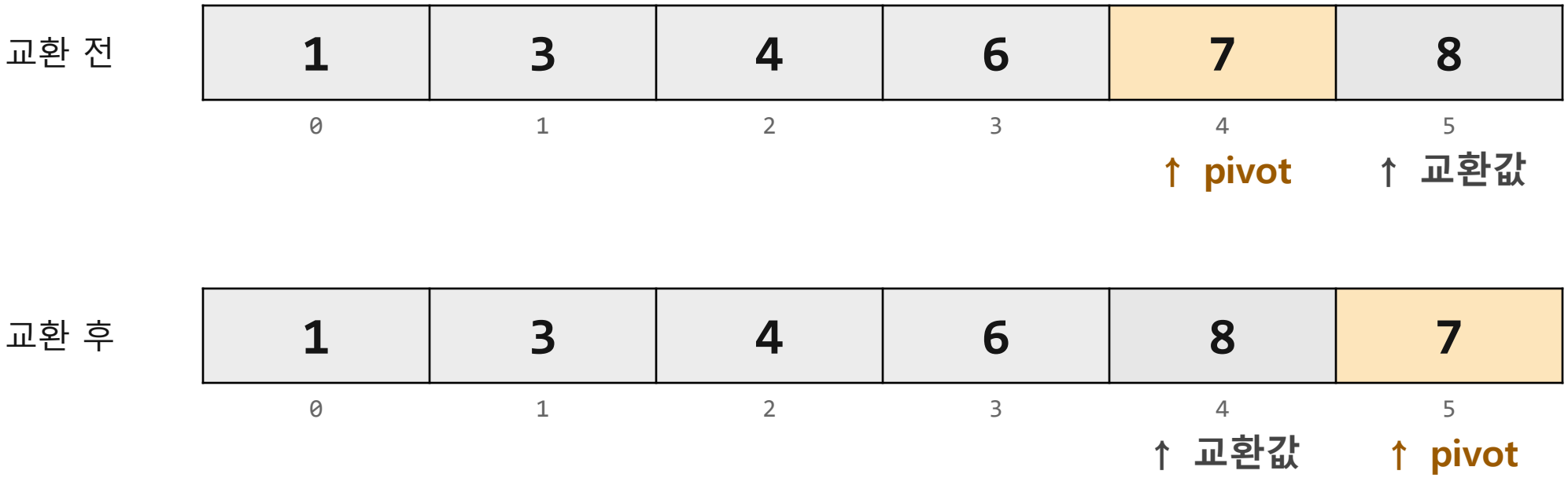

```
quick_sort(a, 3, 2)
quick_sort(a, 4, 5)
```

왼쪽 [1,3,4]는 완료했습니다. 다음 검사 범위는 인덱스 4~5입니다.

오른쪽 [7,8]: 피벗 7의 이동

lo = 4, hi = 5입니다. 가운데 인덱스는 4이고 피벗은 a[4] = 7입니다.
피벗 7과 현재 구간의 마지막 값 a[5] = 8을 교환합니다.

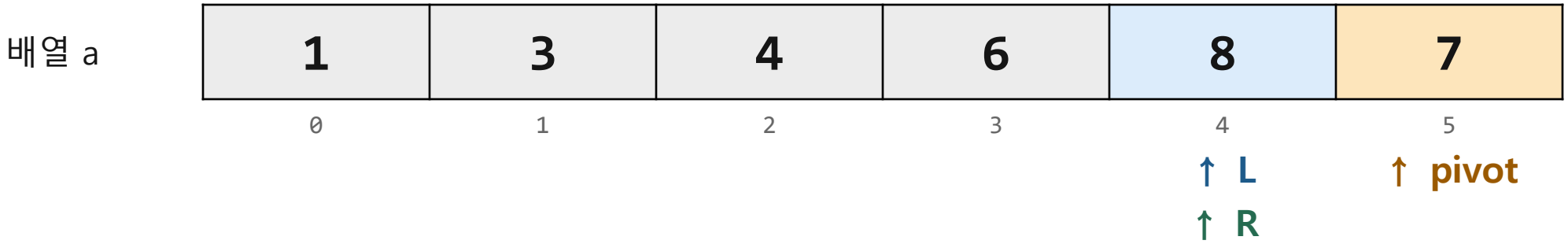
```
a[4], a[5] = a[5], a[4]
```



원래 [7,8]이어도 이 코드의 준비 교환을 그대로 실행합니다.

피벗 7: L = R = 4에서 시작

검사할 값은 a[4] = 8 하나입니다. 피벗 7은 인덱스 5에 둡니다.
8 > 7이므로 L은 멈춥니다. 이어서 R의 조건을 검사합니다.

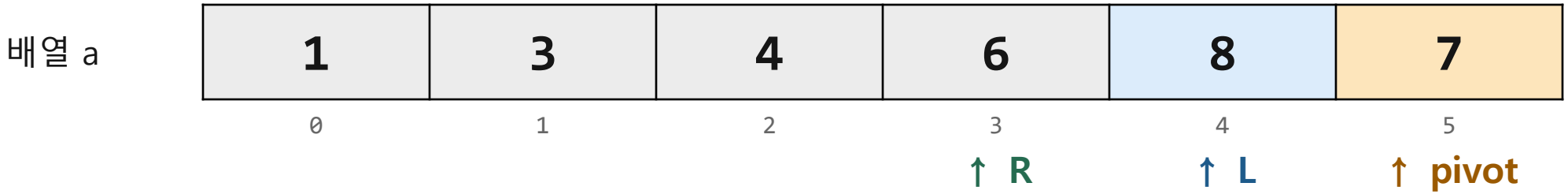



```
pivot = 7
L, R = 4, 4
```

L은 8 > 7에서 멈춥니다. R은 같은 8을 지나 왼쪽으로 이동합니다.

피벗 7: R이 4에서 3으로 이동

a[4] = 8 > 7이므로 R을 3으로 줄입니다. L = 4, R = 3으로 교차합니다.
L ≤ R 조건이 거짓이므로 a[3] = 6은 비교하지 않습니다. 이번 탐색은 끝입니다.




```
L, R = 4, 3
```

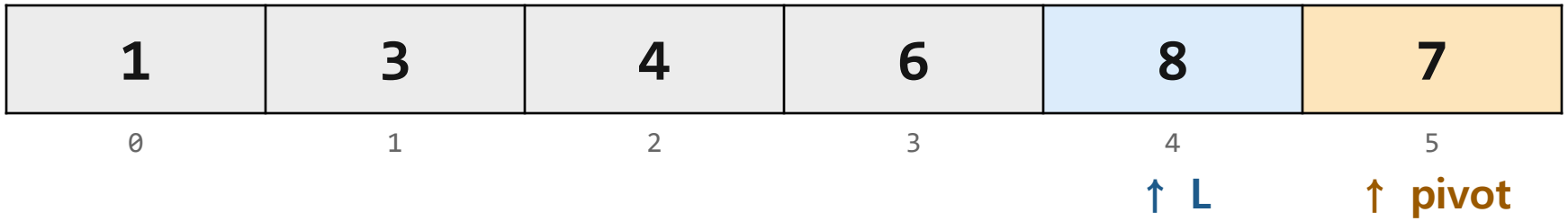
R은 범위 밖으로 이동했지만, 범위 밖의 값 6은 검사하거나 바꾸지 않습니다.

피벗 7과 8 교환: 인덱스 4 확정

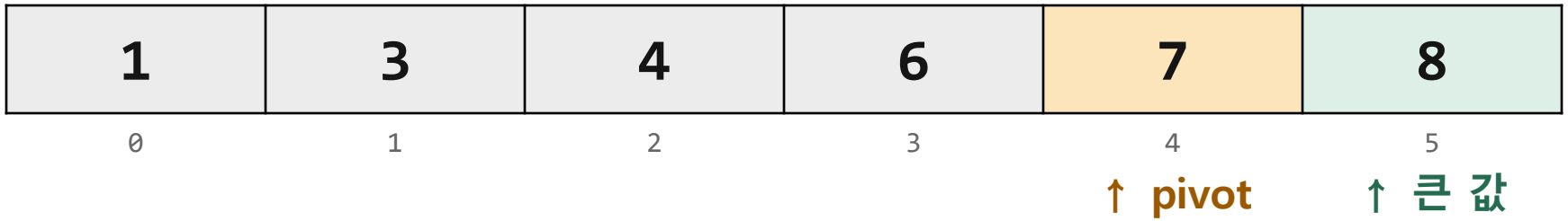
L = 4의 값 8과 hi = 5의 피벗 7을 교환합니다.
피벗 7은 인덱스 4, 큰 값 8은 인덱스 5에 놓입니다.

```
a[4], a[5] = a[5], a[4]
```

교환 전



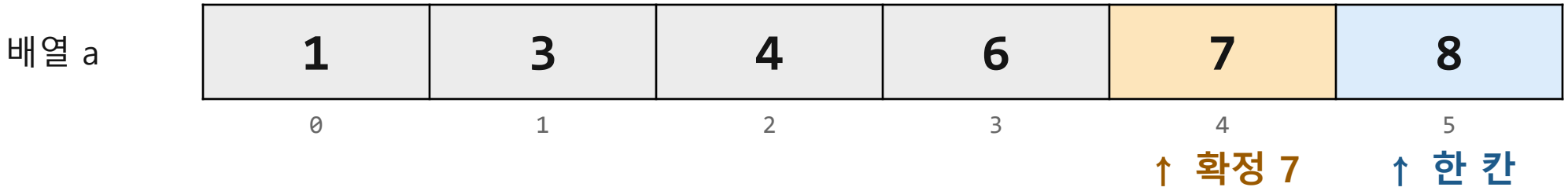
교환 후



partition의 반환값 p = 4입니다. 다음 구간은 [4,3]과 [5,5]입니다.

오른쪽의 남은 호출도 종료

왼쪽 [4,3]은 빈 구간이고, 오른쪽 [5,5]는 값 8 하나입니다.
두 호출 모두 lo ≥ hi에서 끝납니다. 오른쪽 4~5의 정렬도 완료했습니다.

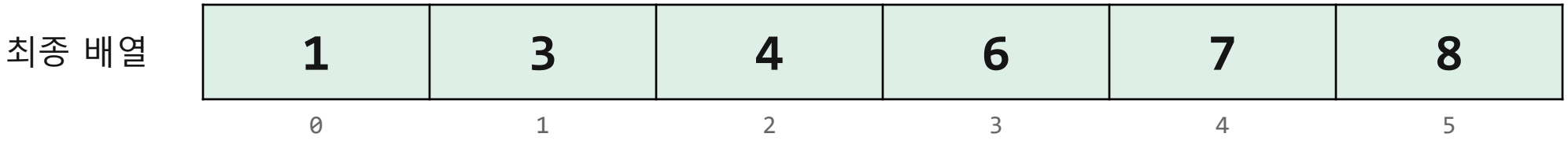



```
quick_sort(a, 4, 3)
quick_sort(a, 5, 5)
```

이 두 호출은 partition 없이 끝납니다. 원래 0~5 호출로 돌아갑니다.

퀵 정렬 완료: 모든 호출 종료

최종 배열은 [1,3,4,6,7,8]입니다. 원래 배열 안에서 정렬했습니다.
왼쪽 호출과 오른쪽 호출이 모두 끝나, 처음 quick_sort(a, 0, 5)도 종료합니다.




```
quick_sort(a, 0, 5)
```

피벗 확정 순서: 6(인덱스 3), 4(인덱스 2), 3(인덱스 1), 7(인덱스 4)

Python: 그림과 같은 partition

```
def partition(a, lo, hi):
    pivot_idx = (lo + hi) // 2
    a[pivot_idx], a[hi] = a[hi], a[pivot_idx]
    pivot = a[hi]
    L, R = lo, hi - 1
    while True:
        while L <= R and a[L] <= pivot:
            L += 1
        while L <= R and a[R] > pivot:
            R -= 1
        if L > R:
            break
        a[L], a[R] = a[R], a[L]
    a[L], a[hi] = a[hi], a[L]
    return L
```

Python: 피벗을 제외한 재귀 호출

```
def quick_sort(a, lo, hi):
    if lo >= hi:
        return
    p = partition(a, lo, hi)
    quick_sort(a, lo, p - 1)
    quick_sort(a, p + 1, hi)

a = [8, 4, 6, 3, 7, 1]
quick_sort(a, 0, len(a) - 1)
print(a)  # [1, 3, 4, 6, 7, 8]
```

C: 양쪽 포인터 partition

```
void swap(int *x, int *y) {
    int t = *x; *x = *y; *y = t;
}
int partition(int a[], int lo, int hi) {
    int m = lo + (hi - lo) / 2;
    swap(&a[m], &a[hi]);
    int pivot = a[hi], L = lo, R = hi - 1;
    while (1) {
        while (L <= R && a[L] <= pivot) L++;
        while (L <= R && a[R] > pivot) R--;
        if (L > R) break;
        swap(&a[L], &a[R]);
    }
    swap(&a[L], &a[hi]);
    return L;
}
```

C: 재귀 호출과 실행

```
/* swap과 partition을 이 함수 앞에 함께 넣는다. */
void quick_sort(int a[], int lo, int hi) {
    if (lo >= hi) return;
    int p = partition(a, lo, hi);
    quick_sort(a, lo, p - 1);
    quick_sort(a, p + 1, hi);
}
/* 실행 예: main 안에서 */
int a[] = {8, 4, 6, 3, 7, 1};
quick_sort(a, 0, 5);
/* a는 {1, 3, 4, 6, 7, 8}이 된다. */
```

같은 값과 빈 구간도 처리한다

이 구현은 피벗 이하를 왼쪽, 초과를 오른쪽에 둔다.
같은 값이 있어도 L이 계속 이동하여 무한 반복을 피한다.
모두 같은 값에서는 한쪽으로 치우쳐 최악 $\Theta(n^2)$가 된다.
빈 배열의 첫 호출은 quick_sort(a, 0, −1), 즉시 종료

모두 같은 값

2	2	2

분할 크기

왼쪽 2개	피벗 1개	오른쪽 0개

파랑 L: 왼쪽 포인터 초록 R: 오른쪽 포인터 주황: 피벗

피벗에 따라 달라지는 시간

분할 한 번은 Θ(n)
균형 분할은 Θ(log n) 단계, 전체 Θ(n log n)
한쪽에 n − 1개씩 남으면 최악 Θ(n²)
서로 다른 값의 무작위 순열에서 평균 Θ(n log n)
가운데 인덱스의 값이 중앙값이라는 보장은 없다.

균형 분할

절반	피벗	절반

치우친 분할

n − 1개	피벗	0개

두 정렬의 차이 확인

병합 정렬: 위치로 반씩 나누고, 정렬된 결과를 병합
퀵 정렬: 피벗 값으로 분할하고, 양쪽을 재귀 정렬
병합은 최악에도 $\Theta(n \log n)$, 퀵은 최악 $\Theta(n^2)$
현재 병합 코드는 안정적, 퀵 코드는 안정성 보장 없음
확인: 퀵의 첫 분할만 끝나면 전체가 정렬되어 있을까?

4.4

최대 구간 합

Maximum Subarray Sum
주가 등락폭 데이터에서 가장 큰 이익을 내는 연속 구간을 찾아라

문제 — 가장 큰 이익을 내는 구간은?

A 회사의 지난 며칠간의 주가 등락폭(변동분) 데이터가 주어졌다. 목표는 가장 큰 이익을 낼 수 있는 연속된 구간을 찾는 것이다.

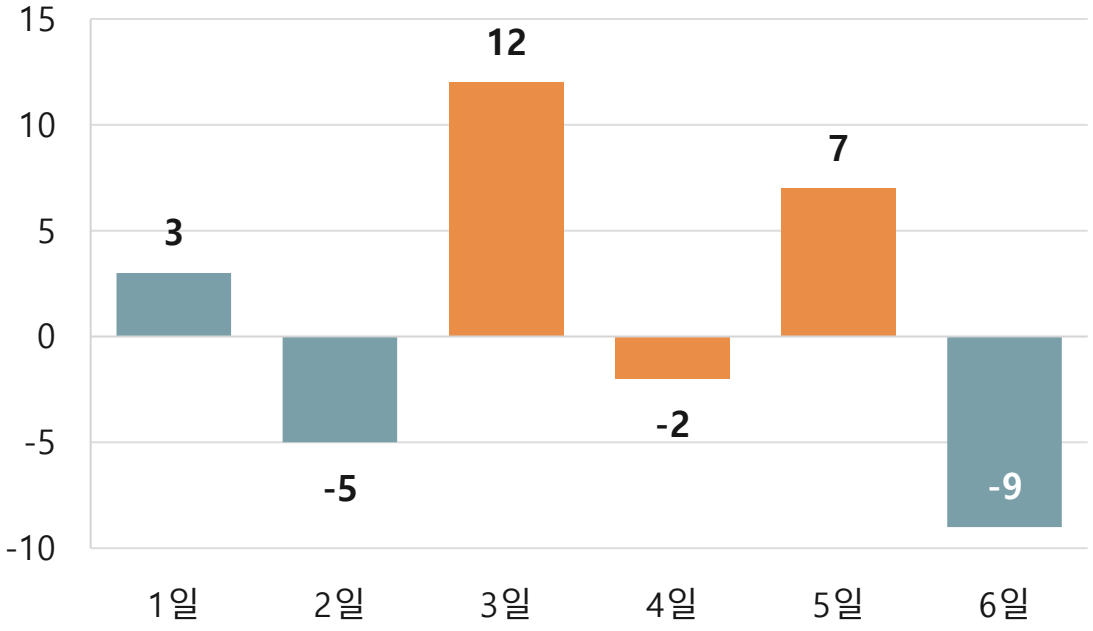
3	-5	12	-2	7	-9
1일	2일	3일	4일	5일	6일



처음부터 끝까지 들고 있었다면? 3 − 5 + 12 − 2 + 7 − 9 = 6

3~5일 차 변동분을 합하면? 12 + (−2) + 7 = 17 ← 최대!

일별 손익 변화



3~5일: 12 − 2 + 7 = 17

직관적인 접근의 한계

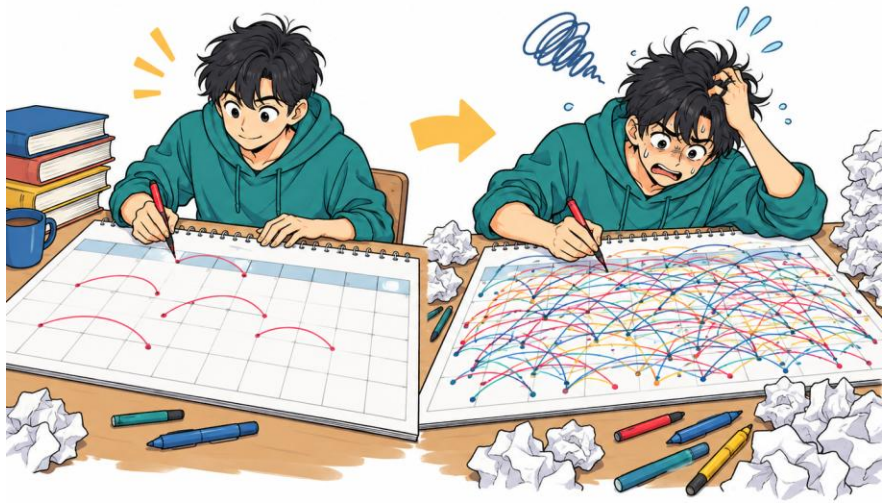
완전 탐색은 가능한 모든 시작점과 끝점을 확인한다. 비어 있지 않은 연속 구간의 개수는 $n(n + 1)/2$이다.

n = 100 → 5,050개
n = 1,000 → 500,500개
n = 100,000 → 5,000,050,000개

시작점을 고정하고 끝점을 늘리며 합을 갱신하면 $O(n^2)$에 모든 구간을 조사할 수 있다.

매 구간의 합을 처음부터 다시 계산하면 $O(n^3)$까지 늘어난다.

구간 수 자체가 많으므로 더 빠른 방법이 필요하다.



분할 정복 전략 — 셋 중 하나에는 반드시 있다!

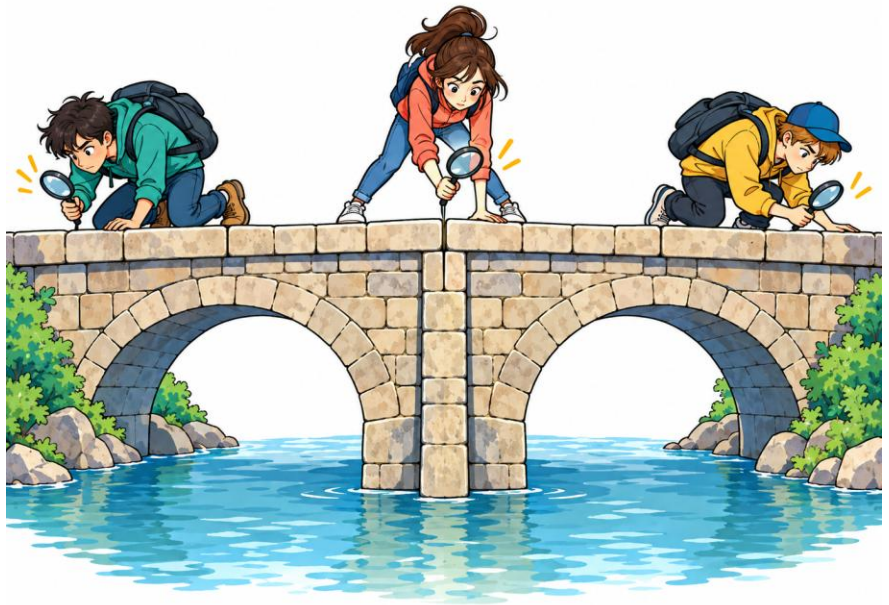
배열을 절반으로 나누면 최대 구간은 다음 세 경우 중 하나에 속한다.

① 왼쪽에만 존재 → 왼쪽 절반을 재귀적으로 해결

② 오른쪽에만 존재 → 오른쪽 절반을 재귀적으로 해결

③ 경계를 가로지름 → 왼쪽 최대 접미 합과 오른쪽 최대 접두 합을 더한다.

정답 = max(leftMax, rightMax, crossMax)
세 경우를 모두 비교하므로 최대 구간을 놓치지 않는다.



'걸친 구간' 처리하기

걸친 구간을 처리하는 부분이 분할 정복 알고리즘에서 가장 까다롭다. 핵심은 "걸친 구간은 반드시 중앙(mid)을 포함해야 한다"는 것이다. 우리가 찾는 것은 왼쪽 동네와 오른쪽 동네를 연결하는 다리 같은 구간이다.



mid에서 왼쪽으로 훑으며 최대 합(leftSum), mid+1에서 오른쪽으로 훑으며 최대 합(rightSum). 둘을 더하면 걸친 구간의 최대 합이다.

최대 구간 합 알고리즘 (메인)

max_subarray.pseudo

```
MaxSubarraySum(A, low, high):
    if low == high then    // 기저 사례: 원소가 하나
        return A[low]
    mid <- floor((low + high) / 2)
    leftMax  <- MaxSubarraySum(A, low, mid)
    rightMax <- MaxSubarraySum(A, mid + 1, high)
    crossMax <- MaxCrossingSum(A, low, mid, high)
    return max(leftMax, rightMax, crossMax)
```

기저 사례
low == high 이면 원소가 하나뿐이므로 그 값이 곧 최대 구간 합이다.

분할
mid를 기준으로 왼쪽·오른쪽 두 개의 절반 문제로 나눈다.

정복
두 절반을 재귀 호출로 해결한다. 문제 크기는 매번 절반이 된다.

결합
걸친 구간을 계산하고, 세 후보 중 최댓값을 돌려준다.

$$T(n) = 2T(n/2) + O(n)$$

최대 구간 합 알고리즘 (걸친 구간)

max_crossing.pseudo

```
MaxCrossingSum(A, low, mid, high):
    leftSum <- -INF
    sum <- 0
    for i <- mid downto low do
        sum <- sum + A[i]
        if sum > leftSum then leftSum <- sum
    rightSum <- -INF
    sum <- 0
    for j <- mid + 1 to high do
        sum <- sum + A[j]
        if sum > rightSum then rightSum <- sum
    return leftSum + rightSum
```

왜 −∞ 로 초기화하는가?

모든 원소가 음수인 배열에서도 올바른 최댓값을 찾기 위해서다. 0으로 초기화하면 "아무것도 안 고르는 것"이 최선이 되어 버린다.

mid를 반드시 포함한다

왼쪽 스캔은 mid에서 출발하고 오른쪽 스캔은 mid+1에서 출발하므로, 두 구간은 항상 경계선을 가로지른다.

비용: O(n)

파이썬 구현 — 걸친 구간 (combine)

max_crossing_sum.py

```
def max_crossing_sum(arr, low, mid, high):
    left_sum = float('-inf')
    current_sum = 0
    for i in range(mid, low - 1, -1):
        current_sum += arr[i]
        if current_sum > left_sum:
            left_sum = current_sum
    right_sum = float('-inf')
    current_sum = 0
    for i in range(mid + 1, high + 1):
        current_sum += arr[i]
        if current_sum > right_sum:
            right_sum = current_sum
    return left_sum + right_sum
```

range의 끝은 포함되지 않는다

mid부터 low까지 거꾸로 돌려면 range(mid, low − 1, −1)로 써야 low가 포함된다.

한 번의 선형 스캔

왼쪽 스캔과 오른쪽 스캔 모두 각 원소를 한 번씩만 훑는다. 따라서 이 함수의 비용은 구간 길이에 비례하는 O(n)이다.

파이썬 구현 — 분할 정복 메인

max_subarray_sum.py

```
def max_subarray_sum(arr, low, high):
    if low == high:
        return arr[low]
    mid = (low + high) // 2
    left_max  = max_subarray_sum(arr, low, mid)
    right_max = max_subarray_sum(arr, mid + 1, high)
    cross_max = max_crossing_sum(arr, low, mid, high)
    return max(left_max, right_max, cross_max)
```

run.py

```
stock_prices = [3, -5, 12, -2, 7, -9, 5, 1]
n = len(stock_prices)
max_profit = max_subarray_sum(stock_prices, 0, n - 1)
print(f"최대 구간 합(최대 이익): {max_profit}")
```

세 개의 후보

왼쪽 절반의 최대, 오른쪽 절반의 최대, 그리고 중앙을 걸친 최대. 파이썬 내장 max()로 셋 중 가장 큰 값을 고른다.

OUTPUT

```
주가 데이터: [3, -5, 12, -2, 7, -9, 5, 1]
최대 구간 합(최대 이익): 17
```

T(n) = O(n log n)

시간 복잡도 분석

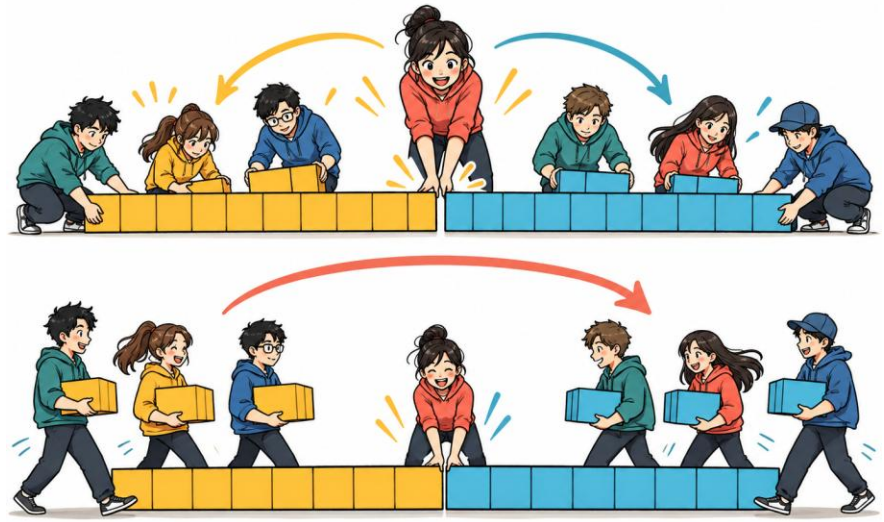
분할 — 중간 인덱스를 계산한다. O(1)

정복 — 절반 크기의 두 문제를 푼다. 2T(n/2)

결합 — 중앙에서 양쪽으로 훑어 경계를 가로지르는 최대 합을 구한다. O(n)

T(n) = 2T(n/2) + O(n)
따라서 T(n) = O(n log n)

병합 정렬과 같은 형태의 점화식이다.



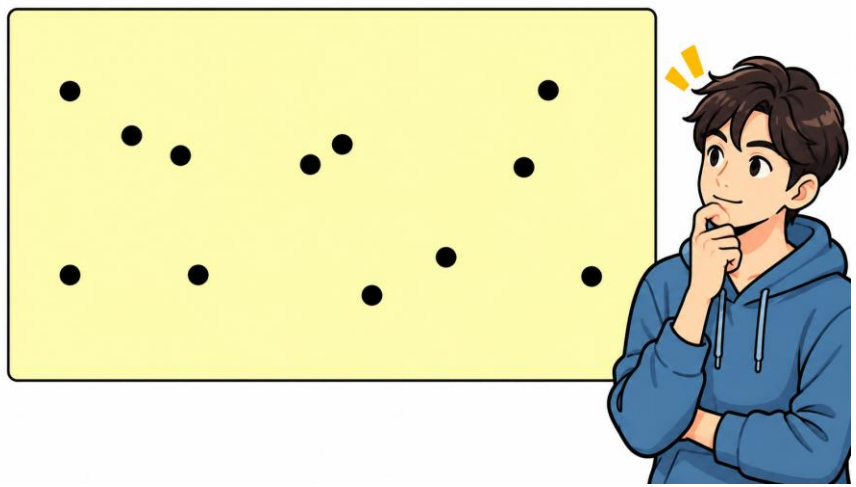
4.5

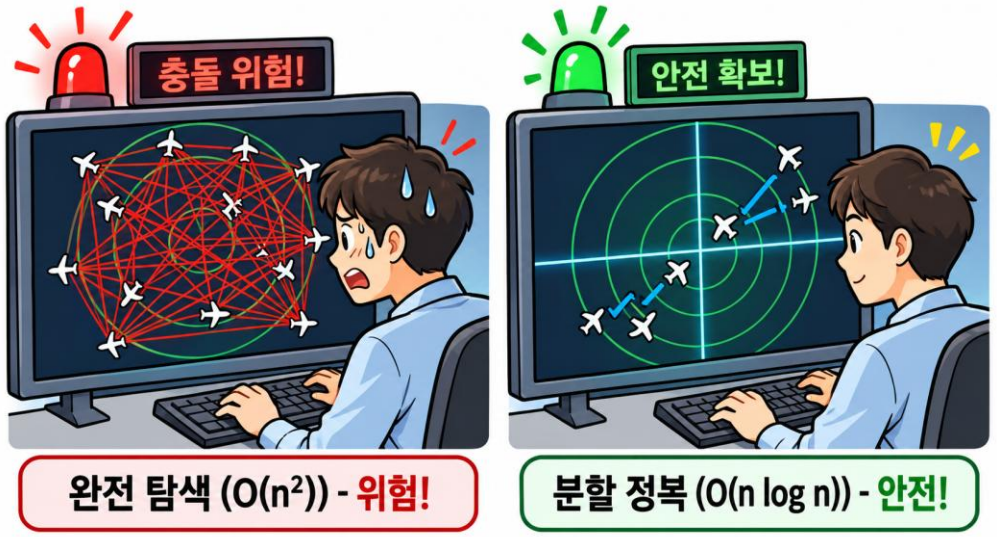
최근접 쌍 문제

Closest Pair of Points
평면 위 n개의 점 중 가장 가까운 두 점을 찾아라

문제 — 평면 위 가장 가까운 두 점

2차원 평면의 n개 점 중 유클리드 거리가 가장 작은 두 점을 찾습니다. 전체 점에서 두 점을 고르는 문제로, 특정 지점의 최근접 이웃을 찾는 문제와 구분합니다.





항공 관제 — 1초가 생명

레이더에 수백 대의 항공기가 잡힌다. 가장 가까운 두 대를 실시간으로 찾아내야 충돌을 막을 수 있다.

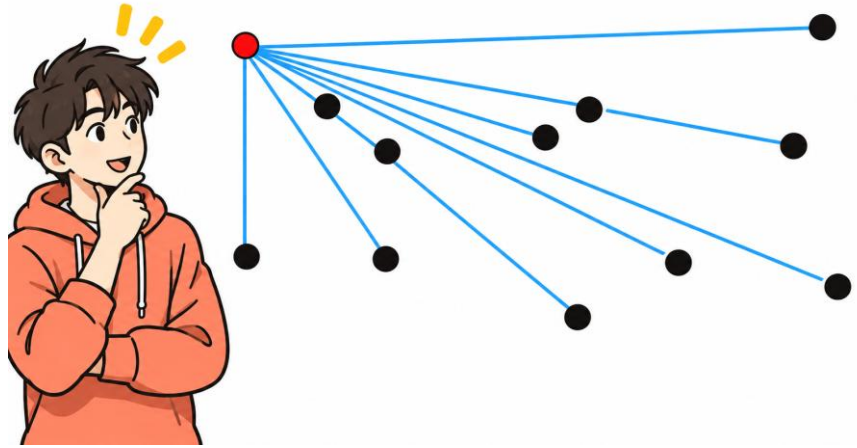
무식한 접근 — 모든 쌍을 다 재본다

가장 단순한 방법은 모든 점의 쌍에 대해 거리를 계산하고 최솟값을 고르는 것이다. 첫 번째 점은 나머지 (n−1)개와, 두 번째 점은 (n−2)개와 비교해야 한다.

$$(n-1) + (n-2) + \cdots + 1 = n(n-1)/2 = O(n^2)$$

brute_force.py

```
import math
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y
def distance(p1, p2):
    return math.hypot(p1.x - p2.x, p1.y - p2.y)
def ClosestPairBruteForce(P):
    min_dist = float('inf')
    for i in range(len(P)):
        for j in range(i + 1, len(P)):
            min_dist = min(min_dist, distance(P[i], P[j]))
    return min_dist
```



점이 10만 개라면?

약 50억 번의 거리 계산이 필요하다.
매 프레임마다 이 계산을 반복할 수는 없다.

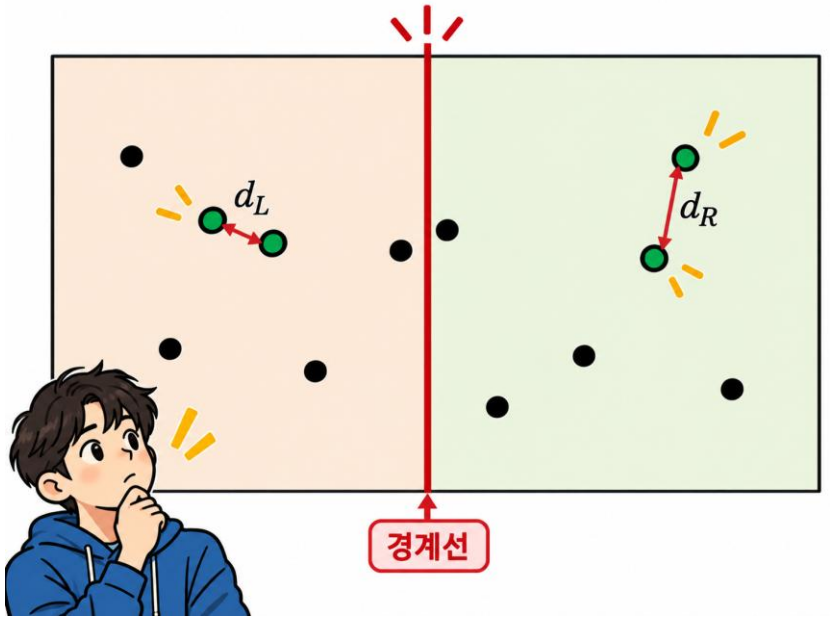
분할 정복 전략 — 평면을 반으로 가른다

x좌표를 기준으로 점들을 정렬한 뒤, 정확히 절반이 되는 지점에 수직선을 긋는다. 그러면 최근접 쌍은 다음 세 경우 중 하나에 속한다.

1. **두 점 모두 왼쪽에**
   재귀 호출로 왼쪽 절반의 최소 거리 dL을 구한다.

2. **두 점 모두 오른쪽에**
   재귀 호출로 오른쪽 절반의 최소 거리 dR을 구한다.

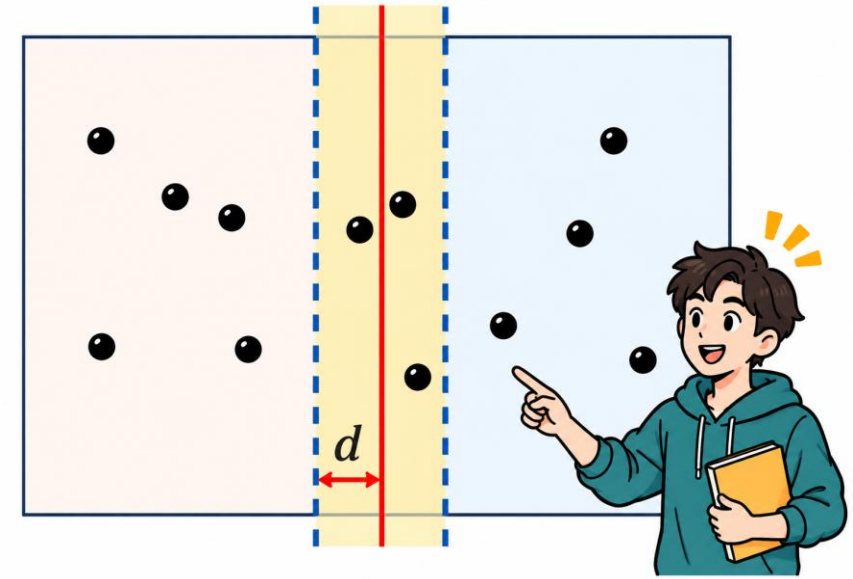
3. **경계선을 사이에 두고**
   한 점은 왼쪽, 다른 점은 오른쪽. 별도로 처리해야 한다.



d = min(dL, dR)을 먼저 구한 뒤, 경계선 근처만 추가로 살펴본다.

경계선의 함정 — '띠(strip)'만 살펴보면 된다

경계선을 걸친 쌍을 모두 검사하면 다시 $O(n^2)$이다. 하지만 이미 $d = \min(dL, dR)$을 알고 있다. d보다 가까운 쌍만 찾으면 되므로, 경계선에서 좌우로 d 이내에 있는 점들만 후보가 된다.



후보를 띠 안으로 좁힌다

경계선에서 수평 거리가 d 이상 떨어진 점은, 어떤 짝을 만나도 거리가 d를 넘는다. 검사할 필요조차 없다.

띠 안의 점을 y좌표로 정렬

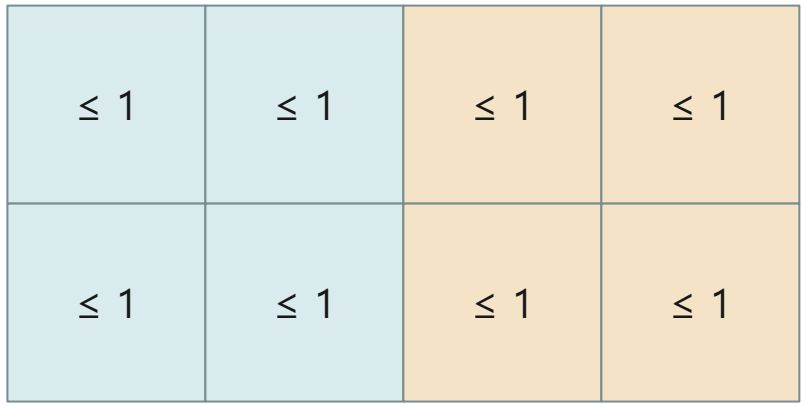
띠 안의 점들을 y좌표 순으로 세우면, 자기보다 y가 d 이상 큰 점은 더 볼 필요가 없다.

그래도 걱정된다

띠 안에 점이 아주 많이 몰려 있으면? 여전히 $O(n^2)$이 아닐까?

증명 — 한 점당 최대 7개만 비교하면 된다

가로 2d



각 칸 d/2 × d/2

8칸: 현재 점을 제외하면 최대 7개

띠 안 점 검사 비용 = O(n) × 7 = O(n)

2d × d 직사각형

띠 안의 한 점 p에서 y 방향으로 d 이내의 영역은 가로 2d, 세로 d인 직사각형이다.

8개의 정사각형 칸

이 직사각형을 (d/2) × (d/2) 크기의 칸 8개로 나눈다.

한 칸에 점은 최대 1개

한 칸의 대각선 길이는 약 0.707d < d. 같은 칸에 두 점이 있으면 이미 d보다 가까운 쌍이 존재해 모순이다.

따라서 최대 7번 비교

직사각형 안의 점은 자기 자신 포함 최대 8개. 즉 한 점당 최대 7번만 비교하면 충분하다.

파이썬 구현 — 분할 정복 본체

closest_pair.py

```
def ClosestPair(P):
    if len(P) <= 3:
        return ClosestPairBruteForce(P)
    mid = len(P) // 2
    Q = P[:mid]
    R = P[mid:]
    d_left  = ClosestPair(Q)
    d_right = ClosestPair(R)
    d = min(d_left, d_right)
    mid_x = P[mid].x
    strip = [p for p in P if abs(p.x - mid_x) < d]
    strip.sort(key=lambda p: p.y)
    for i in range(len(strip)):
        for j in range(i + 1, min(i + 8, len(strip))):
            if distance(strip[i], strip[j]) < d:
                d = distance(strip[i], strip[j])
    return d
```

기저 사례: 점 3개 이하

점이 3개 이하면 나누는 비용이 오히려 크다. 그냥 완전 탐색으로 푼다.

strip 추출

리스트 컴프리헨션 한 줄로 후보를 골라낸다.

min(i + 8, len)

증명 덕분에 안쪽 반복은 상수 횟수로 끝난다.

실행 결과와 시간 복잡도

run.py

```
points = [
    Point(2, 3), Point(3, 4), Point(5, 1),
    Point(12, 10), Point(12, 30), Point(40, 50)
]
points_sorted_by_x = sorted(points, key=lambda p: p.x)
result = ClosestPair(points_sorted_by_x)
print("가장 가까운 두 점 사이의 거리:", result)
```

OUTPUT

```
가장 가까운 두 점 사이의 거리: 1.4142135623730951
```

(2, 3)과 (3, 4) 사이의 거리 $\sqrt{2} \approx 1.414$ 가 정답이다.

분할	x좌표 기준으로 절반	**O(n)**
정복	두 개의 절반 문제 재귀	**2T(n/2)**
결합	strip 정렬 + 상수 회 비교	**O(n log n)**

현재 코드: T(n) = 2T(n/2) + O(n log n)

⇒

O(n log² n)

4.6

쿼드 트리

Quadtree
2차원 공간을 재귀적으로 4등분하는 자료 구조
이미지 압축부터 게임 충돌 감지까지

쿼드 트리의 기본 아이디어

쿼드 트리(quadtree)는 2차원 공간을 재귀적으로 4개의 구역(사분면)으로 분할하여 데이터를 효율적으로 표현하는 자료 구조다. 이진 탐색이 탐색 범위를 1/2씩 줄여나갔다면, 쿼드 트리는 평면을 1/4씩 줄여나간다.

① 확인

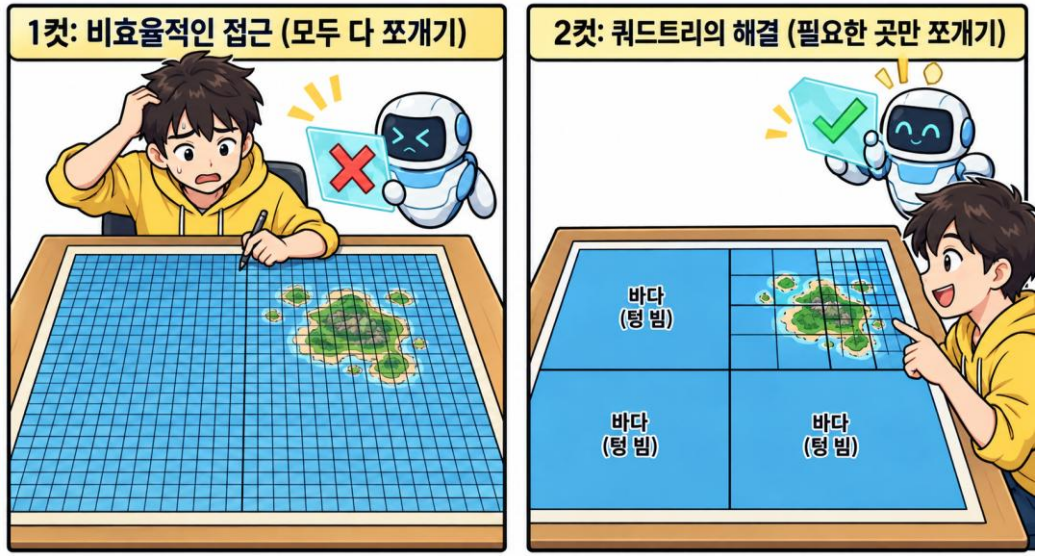
전체 영역을 확인한다.

② 기저 사례

데이터가 '단일 속성'(모두 같은 색이거나 텅 비어 있음)이면 더 이상 나누지 않고 하나의 노드로 표현한다.

③ 재귀 단계

'섞여 있다면' 영역을 정확히 4등분(좌상, 우상, 좌하, 우하)하여 각 부분에 대해 ①을 다시 수행한다.



이론에 그치지 않는다

게임의 물리 엔진(충돌 처리), 포토샵 같은 이미지 소프트웨어(압축)의 핵심 원리다.

응용 ① 흑백 이미지 압축

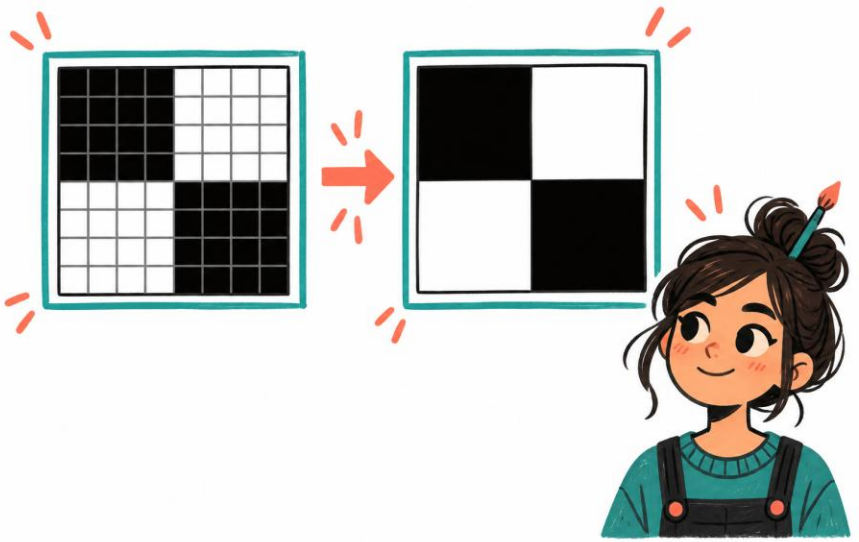
0과 1로 이루어진 영상에서 균일한 영역은 픽셀마다 저장할 필요가 없다.

① 검사: 현재 정사각형이 모두 같은 값인지 확인한다.

② 기저 사례: 모두 0이면 0, 모두 1이면 1을 기록하고 끝낸다.

③ 분할: 값이 섞여 있으면 네 영역으로 나눈다.
좌상 → 우상 → 좌하 → 우하 순서로 재귀 호출하고 결과를 괄호로 묶는다.

균일한 영역이 클수록 짧게 표현된다. 색이 복잡하게 섞이면 압축 효과는 줄어든다.



쿼드 트리 알고리즘 (의사코드)

quadtree_1.pseudo

```
ConstructQuadTree(I, x, y, size):
    initialValue <- I[x][y]
    isUniform <- true
    for i <- x to x + size - 1 do
        for j <- y to y + size - 1 do
            if I[i][j] != initialValue then
                isUniform <- false
    if isUniform then
        return CreateLeafNode(initialValue)
```

quadtree_2.pseudo

```
  else
     half <- size / 2
     root <- CreateInternalNode()
     root.topLeft     <- ConstructQuadTree(I, x,      y,      half)
     root.topRight    <- ConstructQuadTree(I, x,      y+half, half)
     root.bottomLeft  <- ConstructQuadTree(I, x+half, y,      half)
     root.bottomRight <- ConstructQuadTree(I, x+half, y+half, half)
     return root
```

트리 깊이 ≤ log_2(size)

노드 하나가 곧 한 영역

리프 노드는 균일한 영역을, 내부 노드는 4개의 자식으로 쪼개진 영역을 뜻한다. 트리의 깊이는 최대 $\log_2(\text{size})$이다.

파이썬 구현 — 8×8 이미지 압축

quadtree_compress.py

```
def quadtree_compress(x, y, n, image):
    color = image[x][y]
    is_uniform = True
    for i in range(x, x + n):
        for j in range(y, y + n):
            if image[i][j] != color:
                is_uniform = False
                break
        if not is_uniform:
            break
    if is_uniform:
        return str(color)
    half = n // 2
    result = "("
    result += quadtree_compress(x, y, half, image)
    result += quadtree_compress(x, y + half, half, image)
    result += quadtree_compress(x + half, y, half, image)
    result += quadtree_compress(x + half, y + half, half, image)
    result += ")"
    return result
```

example.py

```
example_image = [
    [0,0,0,0, 1,1,1,1],
    [0,0,0,0, 1,1,1,1],
    [0,0,0,0, 1,1,1,1],
    [0,0,0,0, 1,1,1,1],
    [0,0,0,0, 0,0,0,0],
    [0,1,0,0, 0,0,0,0],
    [0,0,0,0, 0,0,0,0],
    [0,0,0,0, 0,0,0,0],
]
print(quadtree_compress(0, 0, 8, example_image))
```

OUTPUT

(01((0001)000)0)

픽셀 64개 → 문자 16개

응용 ② 게임 개발과 충돌 감지

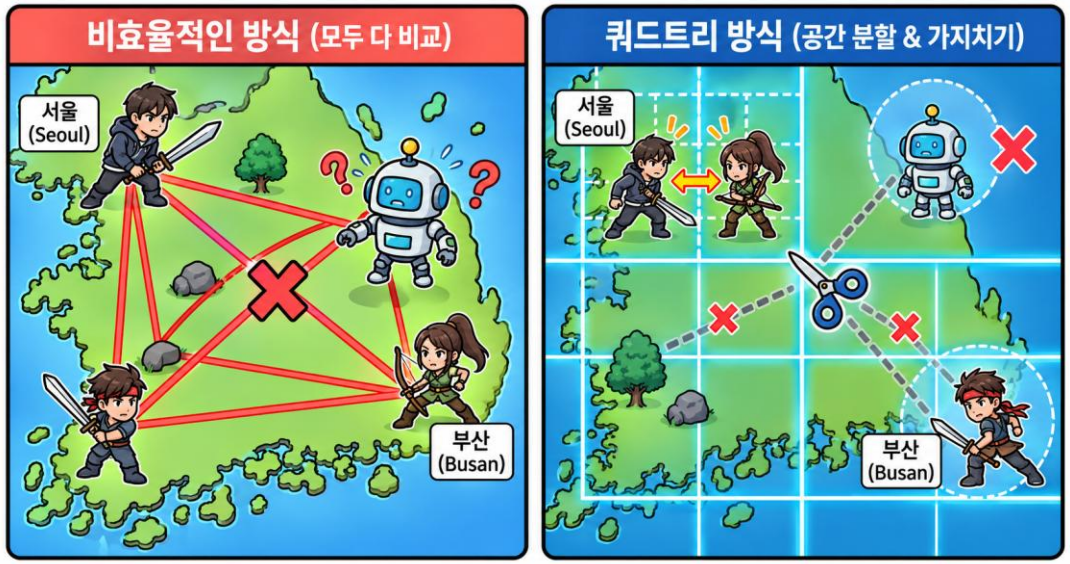
화면 속에는 수십 명의 캐릭터, 수백 발의 총알, 수천 개의 지형지물이 움직인다. 게임 엔진은 이 객체들이 서로 부딪혔는지 판별하는 충돌 감지(collision detection)를 매 프레임(약 0.016초)마다 수행해야 한다.

무식한 접근 — 브루트 포스

$$(n-1) + (n-2) + \cdots + 1 \approx n^2/2$$

객체 1,000개 → 매 프레임 약 50만 번 검사

1. **공간 분할** — 전체 맵을 4개의 사분면으로 나눈다. 한 구역에 오브젝트가 너무 많으면 그 구역을 다시 4개로 쪼갠다.
2. **객체 등록** — 모든 오브젝트는 자신의 위치에 따라 트리의 특정 노드(구역)에 등록된다.
3. **지역적 검사** — 현재 객체와 겹칠 수 있는 영역만 조회한다. 경계를 걸친 객체와 이웃 영역도 포함해 충돌 후보를 빠짐없이 검사한다.

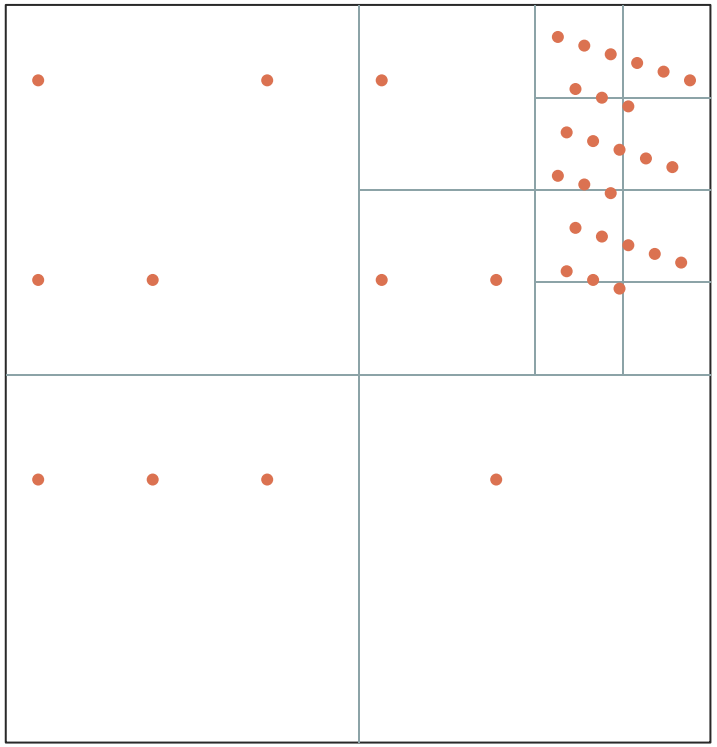


"서로 멀리 떨어져 있는 물체끼리는 충돌 검사를 할 필요가 없다"

성능의 혁신과 LOD (Level of Detail)

쿼드 트리는 지형의 LOD(Level of Detail)를 관리하는 데 활용할 수 있다. 가까운 영역은 작은 격자로, 먼 영역은 큰 격자로 표현한다. 화면에 필요한 세밀함을 판단하는 로직과 함께 사용한다.

가까운 영역	깊게 분할 → 고해상도 렌더링
먼 영역	얕게 분할 → 저해상도 렌더링
결과	같은 프레임 예산으로 훨씬 넓은 세계를 그린다



점이 밀집된 영역을 더 세밀하게 분할

4.7

마스터 정리

Master Theorem
분할 정복 점화식을 재귀 트리 없이 즉시 푸는 강력한 도구

일정한 형태의 점화식을 푸는 정리

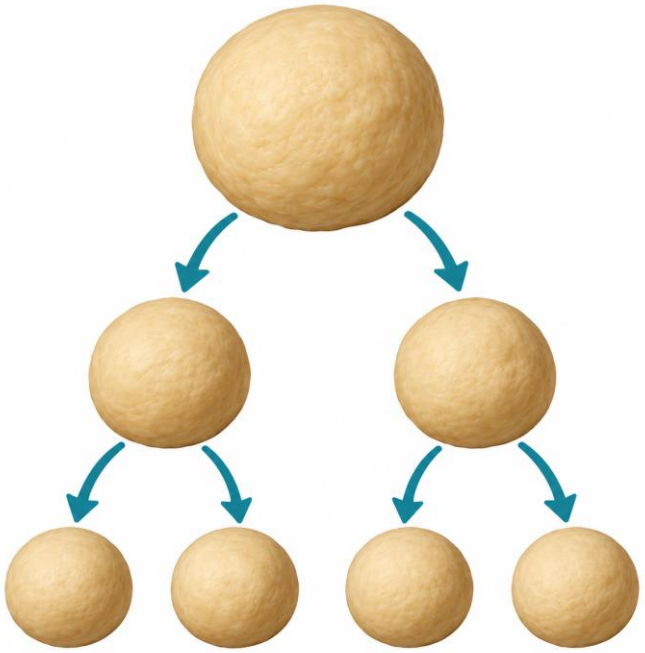
마스터 정리는 다음 형태의 점화식을 분석한다.

$T(n) = a \cdot T(n/b) + f(n)$

a: 재귀 호출의 개수
독립적으로 풀어야 하는 하위 문제의 수이다.

n/b: 각 하위 문제의 크기
한 단계마다 원래 크기의 1/b로 줄어든다.

f(n): 재귀 호출 밖에서 드는 비용
분할, 결합 등 현재 단계의 추가 작업이다.



마스터 정리의 세 가지 경우

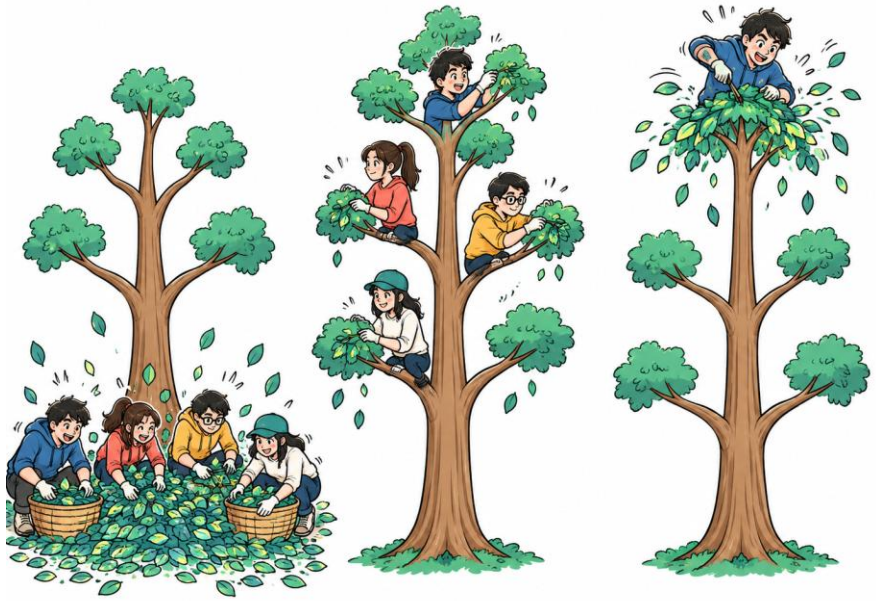
기준 지수 $p = \log_b a$, $\varepsilon > 0$으로 두고 비교한다.

① $f(n) = O(n^{(p-\varepsilon)})$
하위 문제의 비용이 지배 → $T(n) = \Theta(n^p)$

② $f(n) = \Theta(n^p)$
각 층의 비용이 비슷 → $T(n) = \Theta(n^p \log n)$

③ $f(n) = \Omega(n^{(p+\varepsilon)})$
현재 단계의 비용이 지배 → $T(n) = \Theta(f(n))$

③은 $a \cdot f(n/b) \le c \cdot f(n)$, $c < 1$인 정규성 조건도 필요하다. 조건에 맞지 않으면 다른 분석을 사용한다.



적용 ① 이진 탐색

이진 탐색은 한 번 호출할 때마다 문제의 크기가 절반이 되고, 절반 지점(mid)을 계산하는 나눗셈 1번이 필요하다.

$$T(n) = T(n/2) + O(1)$$

`a = 1`	재귀 호출 1회
`b = 2`	문제 크기를 절반으로
`f(n) = O(1)`	mid 계산 시간

마스터 정리 적용

기준값 계산

$n^{(\log_2 1)} = n^{(0)} = 1$

f(n)과 비교

$f(n) = \Theta(1) = \Theta(n^{(0)})$

판정

Case 2 (둘이 같은 정도)

$$T(n) = \Theta(\log n)$$

적용 ② 병합 정렬

병합 정렬은 문제를 2개의 부분 문제로 나누고, 각 부분 문제의 크기는 원본의 1/2이다. 나누고 합치는 데 걸리는 비용은 입력 크기 n에 비례한다.

$$T(n) = 2T(n/2) + cn$$

`a = 2`	2개의 부분 문제로 나눈다
`b = 2`	각 부분 문제 크기는 1/2
`f(n) = cn`	병합 비용은 n에 비례 (c는 상수)

마스터 정리 적용

기준값 계산

$n^{(\log_2 2)} = n^{(1)} = n$

f(n)과 비교

$f(n) = \Theta(n)$

판정

Case 2 (둘이 같은 정도)

$$T(n) = \Theta(n \log n)$$

한눈에 보는 점화식 정리

지금까지 만난 분할 정복 알고리즘들의 점화식을 마스터 정리로 정리하면 다음과 같다.

알고리즘	점화식	a	b	f(n)	경우	시간 복잡도
이진 탐색	T(n) = T(n/2) + O(1)	1	2	O(1)	Case 2	O(log n)
합병 정렬	T(n) = 2T(n/2) + O(n)	2	2	O(n)	Case 2	O(n log n)
최대 구간 합	T(n) = 2T(n/2) + O(n)	2	2	O(n)	Case 2	O(n log n)
최근접 쌍 (y 정렬 유지)	T(n) = 2T(n/2) + O(n)	2	2	O(n)	Case 2	O(n log n)
카라츠바 (§4.7)	T(n) = 3T(n/2) + O(n)	3	2	O(n)	Case 1	O(n^1.585)
스트라센 (§4.8)	T(n) = 7T(n/2) + O(n²)	7	2	O(n²)	Case 1	O(n^2.807)

재귀 호출 횟수 a를 줄이는 것이 가장 강력한 최적화다 — 다음 두 절의 주제다.

4.8

카라츠바 알고리즘

Karatsuba Algorithm (1960)
큰 정수 곱셈의 재귀 호출을 4번에서 3번으로

문제의 분할 (divide)

m은 뒷부분의 자릿수입니다. 큰 수를 앞부분과 뒷부분으로 나눕니다.

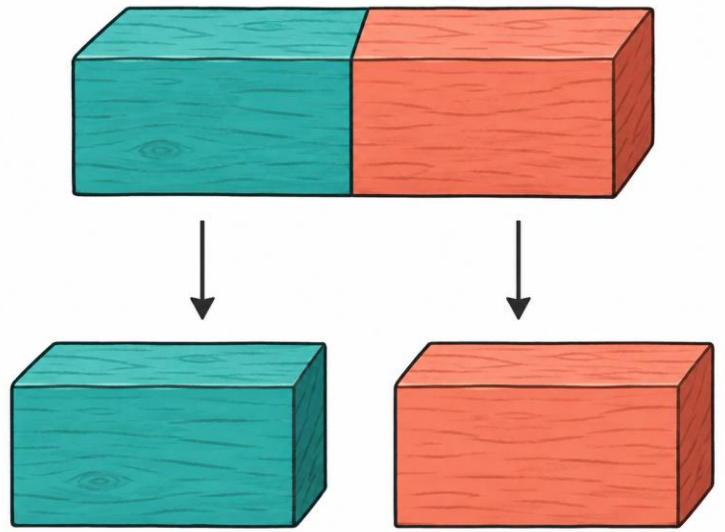
$X = x_1 \cdot 10^m + x_0$
$Y = y_1 \cdot 10^m + y_0$

예: $1234 = 12 \cdot 10^2 + 34$

$X \cdot Y = x_1y_1 \cdot 10^{(2m)} + (x_1y_0 + x_0y_1) \cdot 10^m + x_0y_0$

절반 크기의 곱셈이 네 번 필요합니다.
$T(n) = 4T(n/2) + O(n) = O(n^2)$

다음에는 곱셈 횟수를 세 번으로 줄입니다.



카라츠바의 마법 — 대수적 트릭 (conquer)

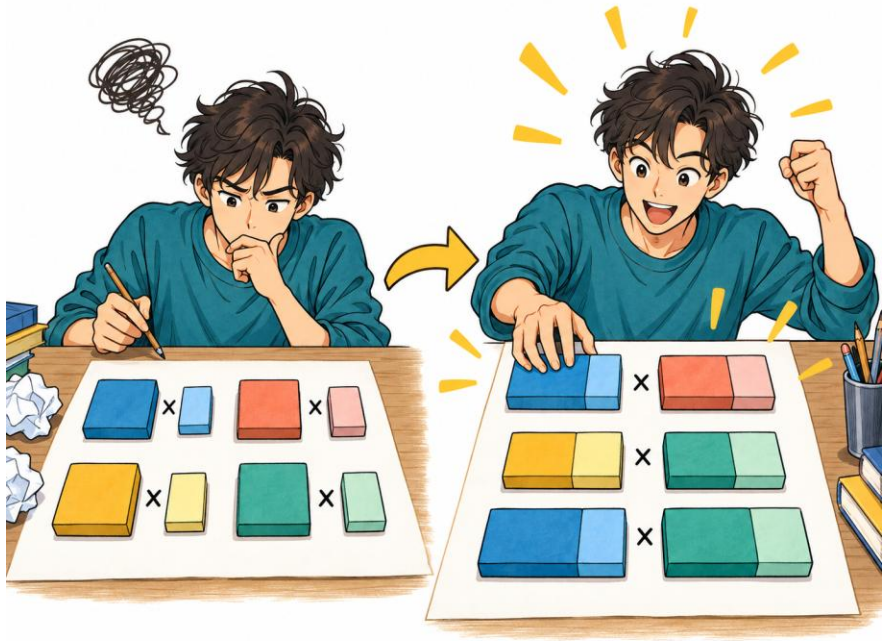
절반 크기의 곱셈을 세 번만 수행한다.

$M_1 = x_1y_1$
$M_2 = x_0y_0$
$M_3 = (x_1 + x_0)(y_1 + y_0)$

교차항은 덧셈과 뺄셈으로 복원한다.
$C = x_1y_0 + x_0y_1 = M_3 - M_1 - M_2$

결합
$X \cdot Y = M_1 \cdot 10^{(2m)} + C \cdot 10^m + M_2$

네 번의 곱셈을 세 번으로 줄이는 것이 카라츠바의 핵심이다.



카라츠바 알고리즘 (의사코드)

karatsuba.pseudo

```
KaratsubaMultiply(X, Y):
    n <- max(length of X, length of Y)
    if n < 2 then
        return X * Y
    m <- floor(n / 2)
    x_high, x_low <- Split(X, m)
    y_high, y_low <- Split(Y, m)
    z2 <- KaratsubaMultiply(x_high, y_high)
    z0 <- KaratsubaMultiply(x_low,  y_low)
    z1_raw <- KaratsubaMultiply(x_high + x_low, y_high + y_low)
    z1 <- z1_raw - z2 - z0
    return z2 * 10^(2m) + z1 * 10^m + z0
```

재귀 호출이 3번뿐

4번이 아니라 3번. 덧셈과 뺄셈이 몇 번 늘어나지만, 이들은 O(n)에 불과하다.

실무에서의 임계값

작은 수에서는 재귀 비용이 더 클 수 있습니다. 기본 곱셈으로 전환할 크기는 구현과 환경에 따라 정합니다.

$$T(n) = 3T(n/2) + O(n)$$

파이썬 구현

karatsuba.py

```
def karatsuba(x, y):
    if x < 10 or y < 10:
        return x * y
    n = max(len(str(x)), len(str(y)))
    m = n // 2
    x_high, x_low = x // (10 ** m), x % (10 ** m)
    y_high, y_low = y // (10 ** m), y % (10 ** m)
    a = karatsuba(x_high, y_high)
    b = karatsuba(x_low, y_low)
    sum_prod = karatsuba(x_high + x_low, y_high + y_low)
    mid = sum_prod - a - b
    return a * (10 ** (2 * m)) + mid * (10 ** m) + b
```

run.py

```
num1 = 12345678901234567890
num2 = 98765432109876543210
print(karatsuba(num1, num2))
```

10 ** m 으로 자르기

몫(//)이 앞부분, 나머지(%)가 뒷부분이 된다.

OUTPUT

```
1219326311370217952
237463801111263526900
```

일반 곱셈 결과와 정확히 일치

큰 정수 연산에 활용

큰 정수 곱셈의 계산량을 줄이는 기본 기법입니다.

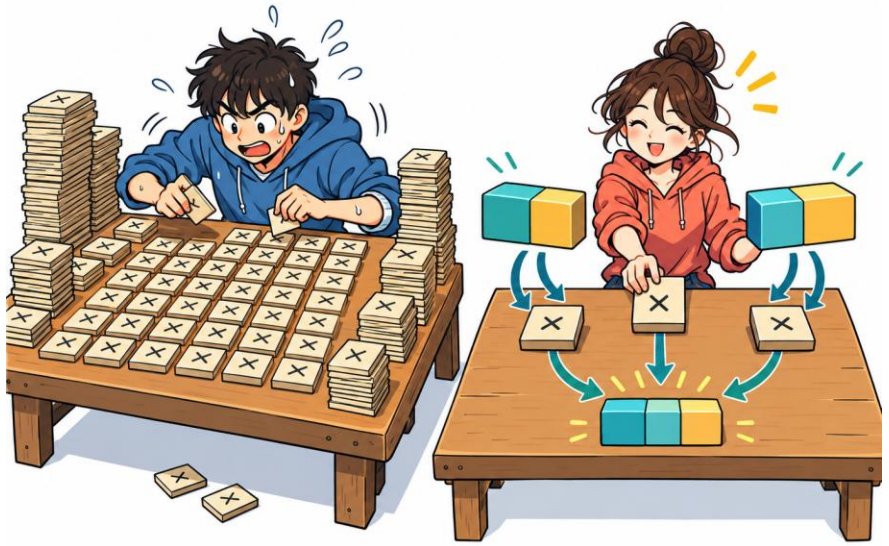
복잡도 분석 — 마스터 정리 적용

문제 크기는 절반: b = 2
재귀 곱셈은 세 번: a = 3
덧셈·뺄셈·자리 이동: O(n)

$T(n) = 3T(n/2) + O(n)$
$= \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$

n = 10,000일 때 성장률 비교
기본 곱셈 n^2: 1억
카라츠바 $n^{\log_2 3}$: 약 219만

비율은 약 46배이다. 상수항과 구현 비용을 생략한 성장률 비교이며 실측 속도는 아니다.



4.9

스트라센 알고리즘

Strassen Algorithm (1969)
행렬 곱셈에서 곱셈 8번을 7번으로

분할 정복의 시도 — 블록 행렬

두 개의 n × n 행렬을 곱하는 표준 알고리즘은 세 겹의 반복문으로 $O(n^3)$이 걸린다. 카라츠바 알고리즘에서 숫자를 절반으로 쪼갰듯이, 행렬도 4등분하여 블록 행렬(block matrix) 형태로 생각해보자.

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \times \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

각 결과 블록은 두 번의 블록 곱셈을 필요로 한다 — 모두 합쳐 8번

$$C_{11} = A_{11}B_{11} + A_{12}B_{21} \qquad C_{12} = A_{11}B_{12} + A_{12}B_{22}$$

$$C_{21} = A_{21}B_{11} + A_{22}B_{21} \qquad C_{22} = A_{21}B_{12} + A_{22}B_{22}$$

아무런 개선이 없다 — $T(n) = 8T(n/2) + O(n^2) \Rightarrow \Theta(n^3)$

블록 곱셈이 8번, 블록 덧셈이 $O(n^2)$. 마스터 정리 Case 1에 의해 $n^{\log_2 8} = n^3$ — 원래 알고리즘과 똑같다.

스트라센의 발견 — "8번이 아니라 7번이면 된다"

1969년, 독일의 수학자 폴커 스트라센(Volker Strassen)은 이 견고해 보이는 벽을 허무는 방법을 찾아냈다. 카라츠바가 그랬던 것처럼, 행렬의 덧셈과 뺄셈을 조금 복잡하게 조합하면 곱셈 횟수를 8번에서 7번으로 줄일 수 있음을 발견했다.

7번의 블록 곱셈

$$M_1 = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$M_2 = (A_{21} + A_{22})\, B_{11}$$

$$M_3 = A_{11}(B_{12} - B_{22})$$

$$M_4 = A_{22}(B_{21} - B_{11})$$

$$M_5 = (A_{11} + A_{12})\, B_{22}$$

$$M_6 = (A_{21} - A_{11})(B_{11} + B_{12})$$

$$M_7 = (A_{12} - A_{22})(B_{21} + B_{22})$$

결과 블록 조립 (combine)

$$C_{11} = M_1 + M_4 - M_5 + M_7$$

$$C_{12} = M_3 + M_5$$

$$C_{21} = M_2 + M_4$$

$$C_{22} = M_1 - M_2 + M_3 + M_6$$

곱셈 1번을 덧셈 여러 번과 맞바꾼다

덧셈·뺄셈은 $O(n^2)$에 불과하지만, 곱셈은 재귀 호출이다. 재귀 호출 횟수 a를 줄이는 것이 압도적으로 이득이다.

복잡도 분석과 두 알고리즘의 공통점

$$T(n) = 7T(n/2) + O(n^{(2)}) \quad \Rightarrow \quad \Theta(n^{(\log_2 7)}) \approx \Theta(n^{(2.807)})$$

	카라츠바 (§4.7)	스트라센 (§4.8)
대상	큰 정수의 곱셈	n × n 행렬의 곱셈
소박한 분할	곱셈 4번	블록 곱셈 8번
개선 후	곱셈 3번	블록 곱셈 7번
점화식	$T(n) = 3T(n/2) + O(n)$	$T(n) = 7T(n/2) + O(n^2)$
시간 복잡도	O(n^1.585)	O(n^2.807)
기존 방법	$O(n^2)$	$O(n^3)$

두 알고리즘의 공통 교훈 — 값싼 덧셈을 늘려서라도 비싼 재귀 호출 횟수 a 를 줄여라.

4.10

코딩 테스트

분할 정복으로 실전 문제 두 개를 직접 풀어본다

색종이 만들기

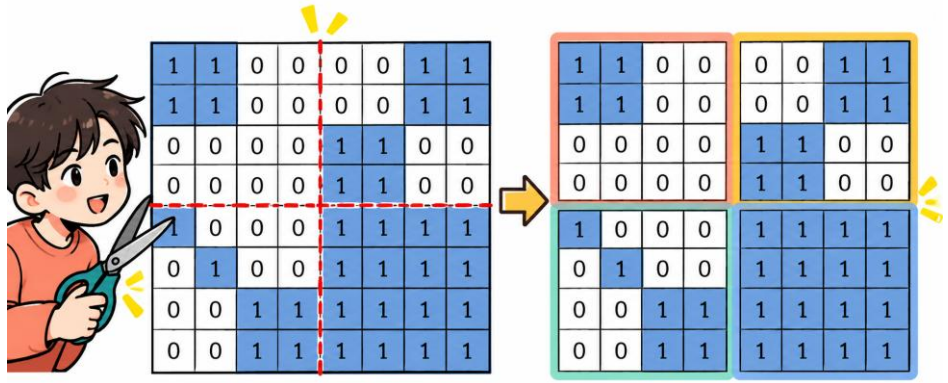
여러 개의 정사각형 칸(N × N)으로 이루어진 종이가 있다. 각 칸은 하얀색 아니면 파란색이다. 전체 종이가 모두 같은 색이 아니면, 가로와 세로의 중앙을 잘라 똑같은 크기의 네 개의 N/2 × N/2 색종이로 나눈다. 이 과정을 잘라진 종이가 모두 같은 색이 되거나 한 칸이 될 때까지 반복한다.

최종적으로 하얀색 색종이와 파란색 색종이가 각각 몇 장인지 구하라.

쿼드 트리와 완전히 동일한 구조

검사(check) → 균일하면 카운트, 아니면 4등분

한 칸(n = 1)은 언제나 균일하므로 재귀는 반드시 끝난다



입력 (N = 8)

`1 1 0 0 0 0 1 1  …  0 0 1 1 1 1 1 1`

출력

하얀색 9장, 파란색 7장

풀이 — 재귀적으로 4등분하며 세기

solve.py

```
import sys
sys.setrecursionlimit(10 ** 6)
def solve(x, y, n):
    global white_cnt, blue_cnt
    color = paper[x][y]
    for i in range(x, x + n):
        for j in range(y, y + n):
            if paper[i][j] != color:
                nn = n // 2
                solve(x,      y,      nn)
                solve(x,      y + nn, nn)
                solve(x + nn, y,      nn)
                solve(x + nn, y + nn, nn)
                return
    if color == 0:
        white_cnt += 1
    else:
        blue_cnt += 1
```

run.py

```
N = 8
white_cnt = blue_cnt = 0
solve(0, 0, N)
print(f"하얀색 색종이: {white_cnt}")
print(f"파란색 색종이: {blue_cnt}")
```

OUTPUT

```
하얀색 색종이: 9
파란색 색종이: 7
```

return 위치가 핵심

다른 색을 만나는 즉시 4등분 후 return 해야 한다. 그러지 않으면 아래의 카운트 코드까지 실행되어 잘못된 답이 나온다.

곱셈 — A^B mod C 를 눈 깜짝할 사이에

세 개의 자연수 A, B, C가 주어진다. A를 B번 곱한 수, 즉 A^B를 C로 나눈 나머지를 구하라. 문제는 A와 B가 최대 2,147,483,647(약 21억)에 달하는 매우 큰 수라는 점이다. 단순한 반복으로는 21억 번의 곱셈이 필요하다.

지수를 절반으로

A^B = A^(B/2) × A^(B/2). 절반만 계산하면 나머지 절반은 공짜다.

B가 홀수라면

A^B = A^(B/2) × A^(B/2) × A. 남는 A 하나를 곱해준다.

모듈러 성질 활용

(a × b) mod c = ((a mod c) × (b mod c)) mod c. 매 단계 나머지를 취해 수가 커지지 않게 한다.

$$T(n) = T(n/2) + O(1) \quad \Rightarrow \quad \Theta(\log B)$$

예제

입력

```
10 11 12
```

출력

```
4
```

21억 번의 곱셈이
약 31단계로 줄어든다.
($\log_2$ 2,147,483,647 ≈ 31)

풀이 — 분할 정복 거듭제곱 (fast power)

power.py

```
def power(a, b, c):
    if b == 0:
        return 1 % c
    half = power(a, b // 2, c)
    result = (half * half) % c
    if b % 2 == 1:
        result = (result * a) % c
    return result
```

run.py

```
A, B, C = map(int, "10 11 12".split())
print(power(A, B, C))
```

OUTPUT 4

재귀 호출 추적

power(10, 11, 12)	b = 11 (홀수)
power(10, 5, 12)	b = 5 (홀수)
power(10, 2, 12)	b = 2 (짝수)
power(10, 1, 12)	b = 1 (홀수)
power(10, 0, 12)	return 1 % c

깊이는 $\log_2 B$

B = 21억이어도 재귀 깊이는 31에 불과하다.

4장 정리

분할 정복의 3단계

분할(divide) → 정복(conquer) → 결합(combine). 기저 사례에 도달할 때까지 재귀적으로 나눈다.

결합이 알고리즘의 핵심

병합 정렬은 나누기가 쉽고 합치기가 어렵다. 반대로 퀵 정렬은 나누기가 어렵고 합치기가 쉽다.

기하 문제에도 통한다

최대 구간 합과 최근접 쌍 모두 "경계를 걸친 경우"만 따로 처리하면 O(n log n)이 된다.

공간도 분할한다

쿼드 트리는 평면을 1/4씩 재귀 분할한다. 이미지 압축, 충돌 감지, LOD의 기반이다.

마스터 정리

T(n) = aT(n/b) + f(n) 형태와 각 경우의 적용 조건을 확인한 뒤 비용의 크기를 비교합니다.

재귀 호출 횟수 a를 줄여라

카라츠바(4→3)와 스트라센(8→7)은 값싼 덧셈으로 비싼 곱셈을 대체했다.

Q & A

질문을 나누어 봅시다.
큰 질문도 작게 쪼개면 답할 수 있습니다.